High-Performance Systems Programming

Lecture 1 – Introduction

Dr. Marina Kogan-Sadetsky

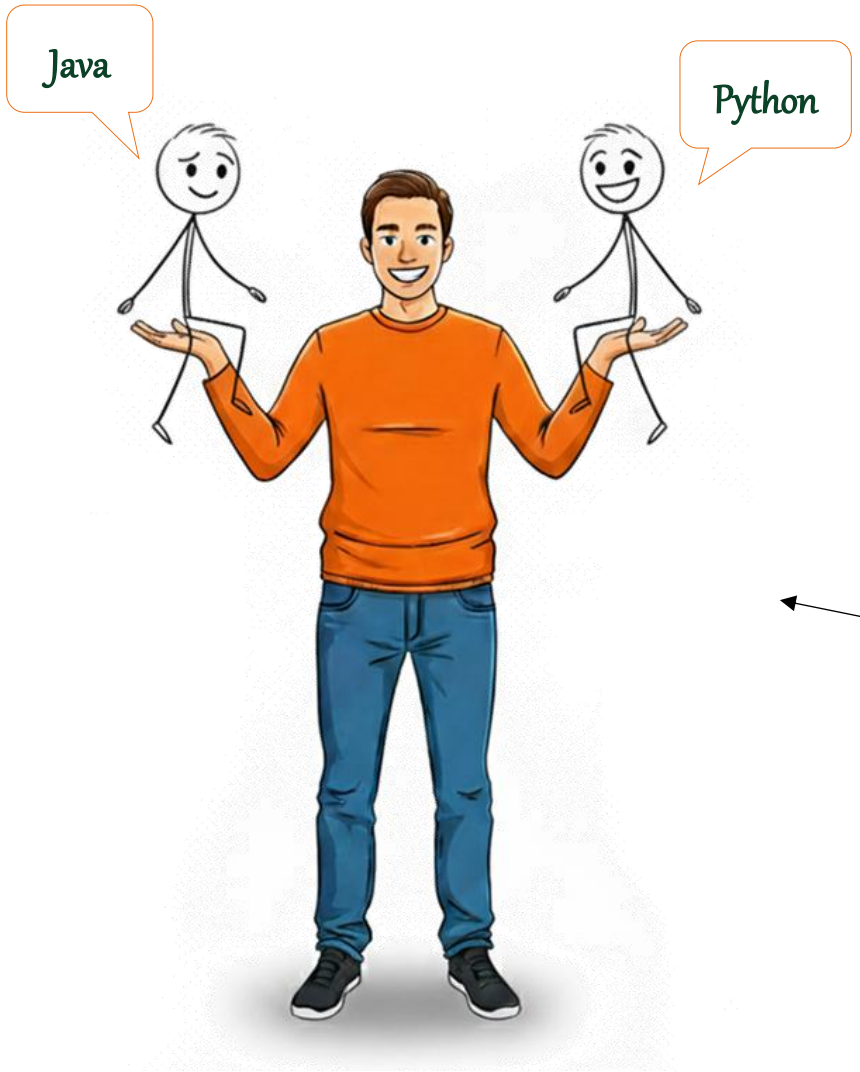


Welcome to High-Performance Systems Programming course!

From C and Memory to Multicore and GPU

C → Memory → Cache / CPU → Multicore → GPU

Low level programming? Why Bother??



Java programmer (you)

Python programmer (you)

Why Go Low Level?

Python / Java
High-level abstractions

↓

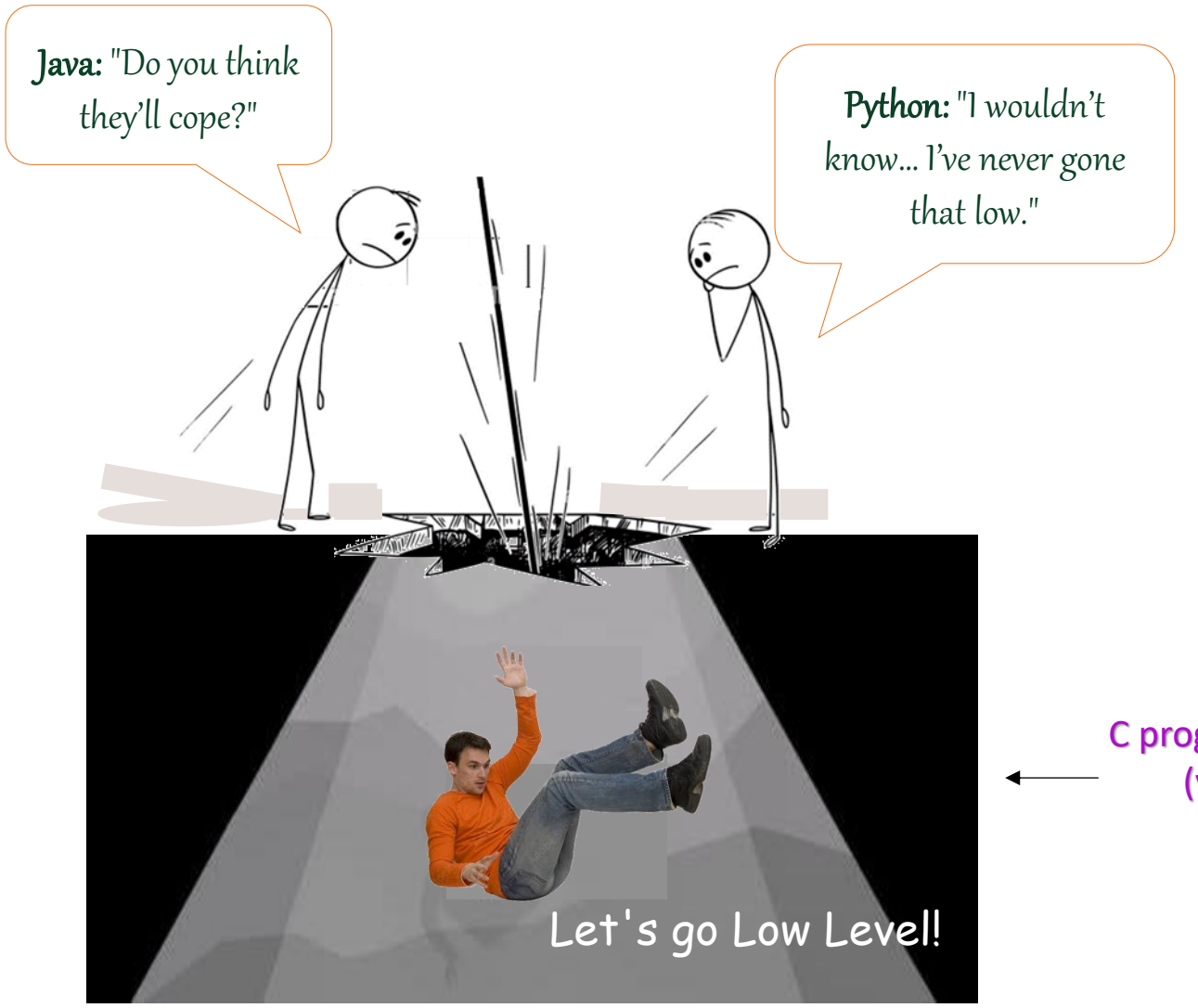
C
Memory, layout, control

↓

Hardware / OS
What actually happens

To write fast and reliable code, we need to understand what abstractions hide.

Low level programming? Why Bother??



C programmer (you)

Low-level details can change performance

even when the algorithm does not change.

There Is No Magic

Every abstraction is implemented by something underneath.

Performance starts where abstractions meet reality.



Theory Vs Reality

Theory vs Reality – example #1

- Is $x^2 \geq 0$?

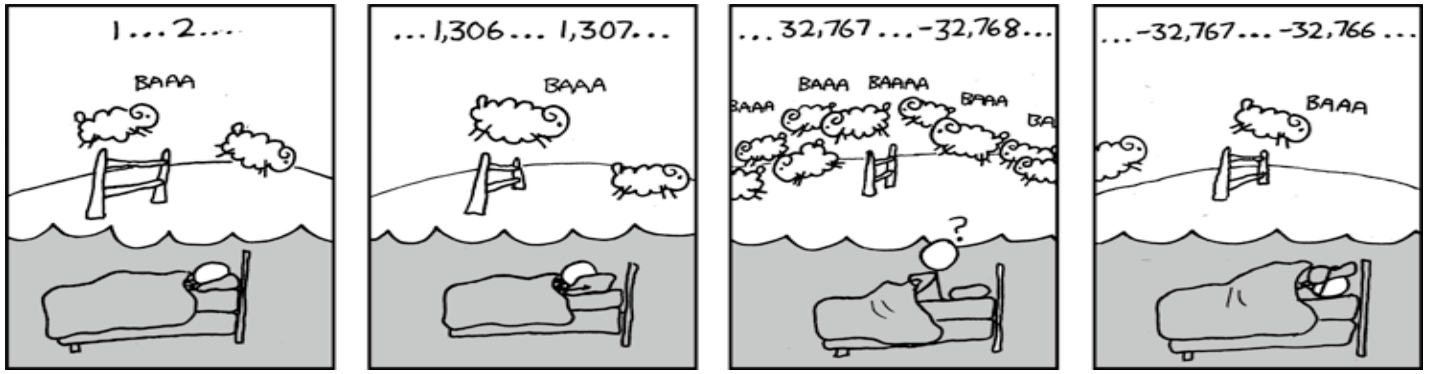
Theory vs Reality – example #1

- Is $x^2 \geq 0$?

לא (תמיד) ניתן לסמוך על כך שמה שאנחנו יודעים בתאוריה מתקיים גם במחשב. איך נתמודד עם זה בתוכניות שלנו ?

This is not bug, but inherent limitations to the representation.

1410065407 * 1410065407 → -1158137855



Theory vs Reality – example #2

- Is $(x + y) + z = x + (y + z)$?

Theory vs Reality – example #2

- Is (x + y) + z = x + (y + z) ?

(1.0E+20 + (-1.0E+20)) + 3.14 → 3.14

but

1.0E+20 + ((- 1.0E+20) + 3.14) → 0.0

לא (תמיד) ניתן לסמוך על כך שמה שאנחנו יודעים בתאוריה מתקיים גם במחשב. איך נתמודד עם זה בתוכניות שלנו ?

This is not bug, but inherent limitations to the representation.

Theory vs Reality – example #3


```
int i,j;
for (i = 0; i < 2049; i++)
  for (j = 0; j < 2049; j++)
     B[i][j] = A[i][j];
```

```
  int i,j;
  for (j = 0; j < 2049; j++)
    for (i = 0; i < 2049; i++)
      B[i][j] = A[i][j];
```

A computer has several types of memory. We will learn how to write code that makes the most efficient use of all these memory types.

This code is slower than the first code. Why ?

```
int i,j;
for (i = 0; i < 2049; i++)
   for (j = 0; j < 2049; j+=3){
      B[i][j] = A[i][j];
      B[i][j+1] = A[i][j+1];
      B[i][j+2] = A[i][j+2];
   }
```

This code may be faster than the first code. Why ?

By understanding how the CPU works, we will learn how to write code that makes the most efficient use of it.

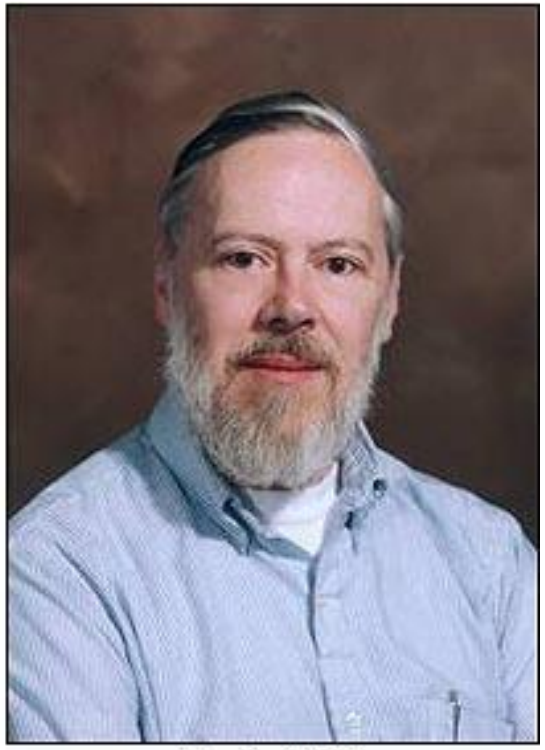
What Will We Learn to Control?



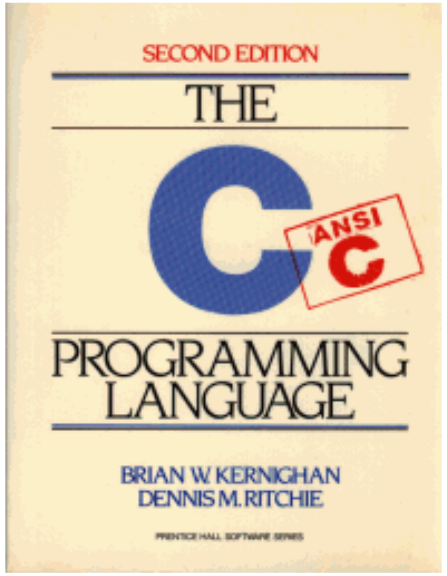
The closer we get to the hardware,
the more control — and responsibility — we get.

Why C?

C is an **efficient** and **portable** language that allows **high-level writing** and the ability to get down to bit-level details.



Dennis Ritchie



1972, Bell Labs

Why C?

More abstraction — More control

trade-off

High-Level Languages

Python / Java

- automatic memory management
- objects & abstractions
- implementation details hidden

C

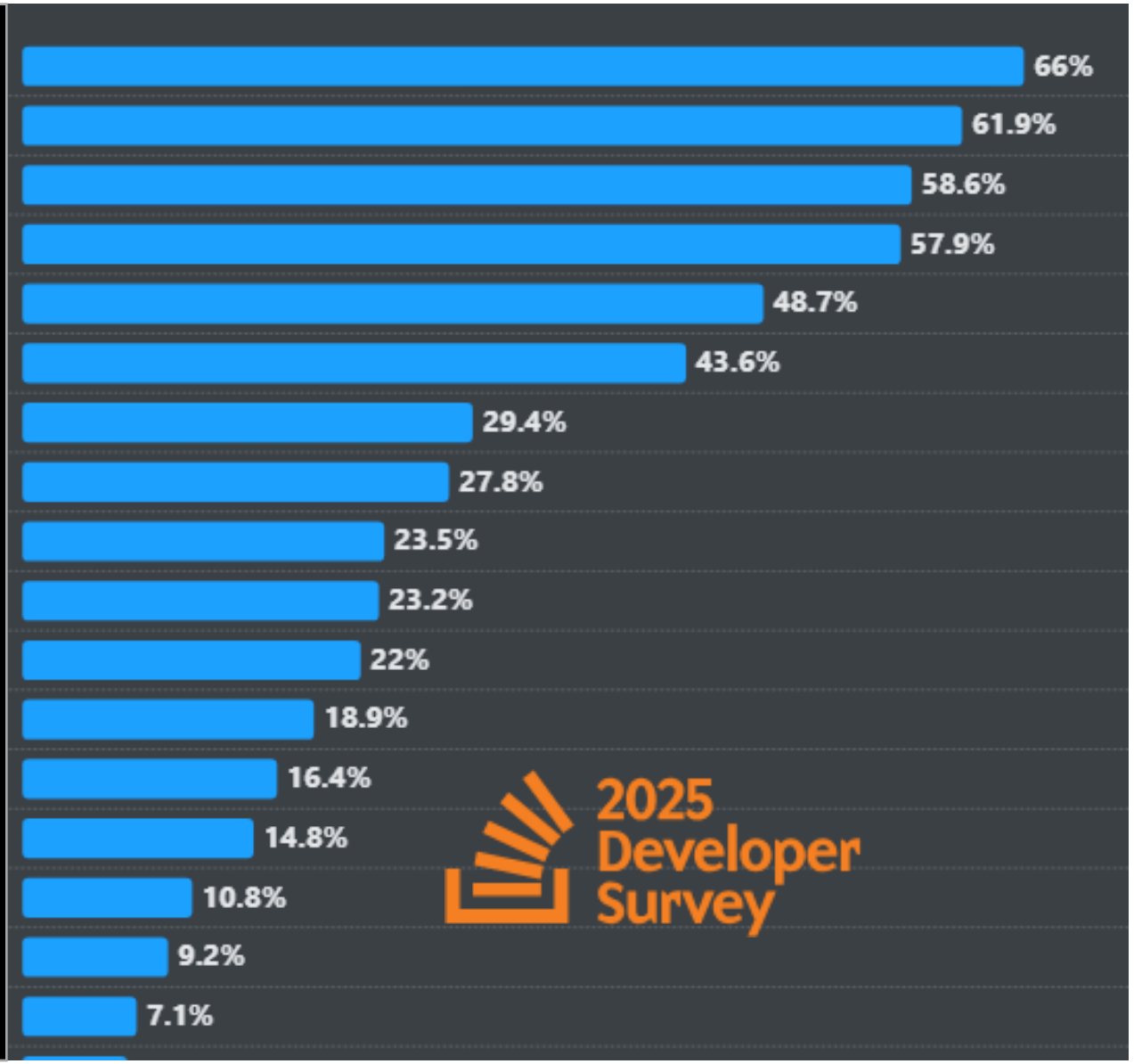
our systems language

- explicit memory management
- direct access to addresses
- control over data representation
- close interaction with OS / hardware

C is not the goal — C is our window into the machine.

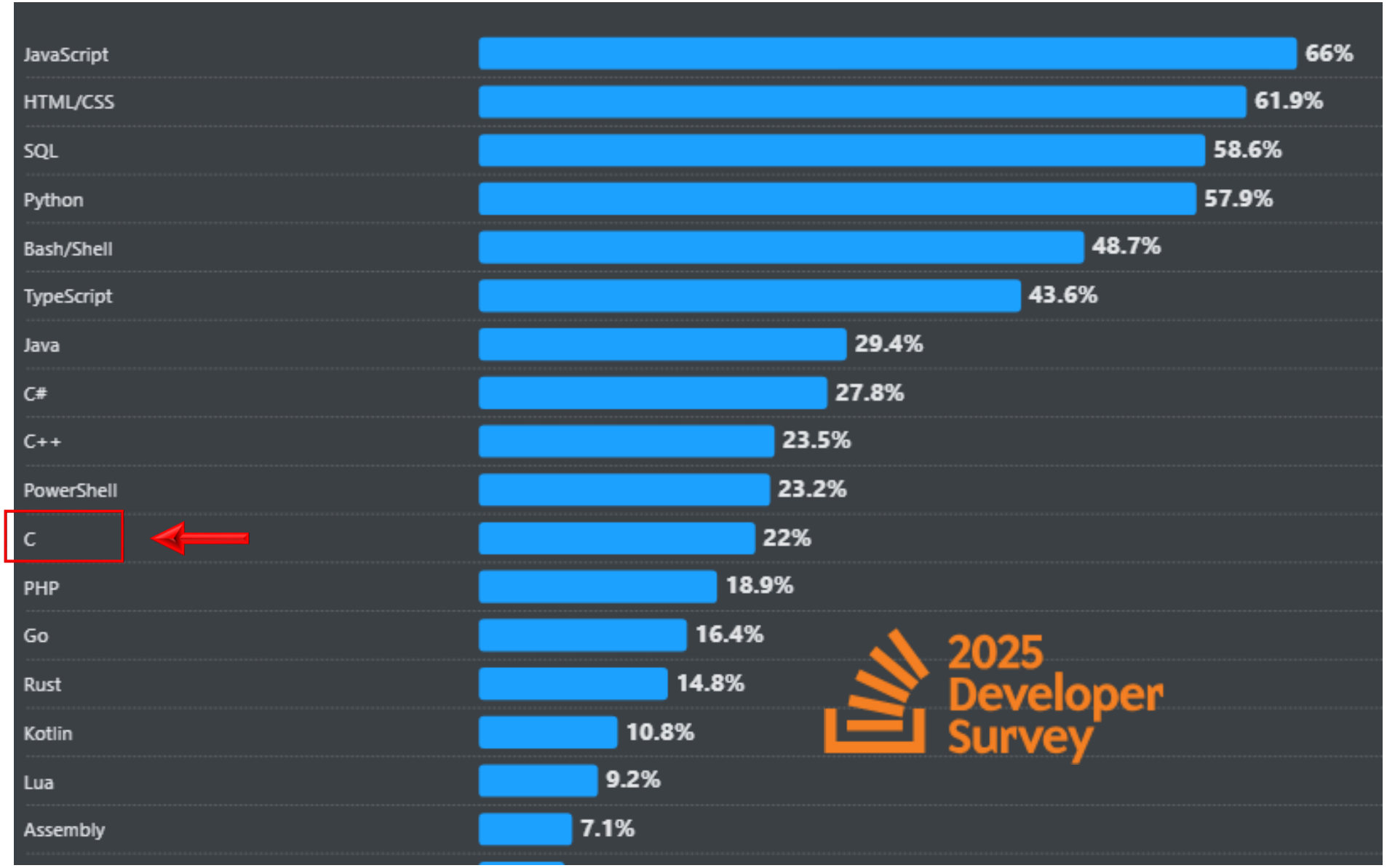
Why C?

?



Guess which line contains C

Why C?

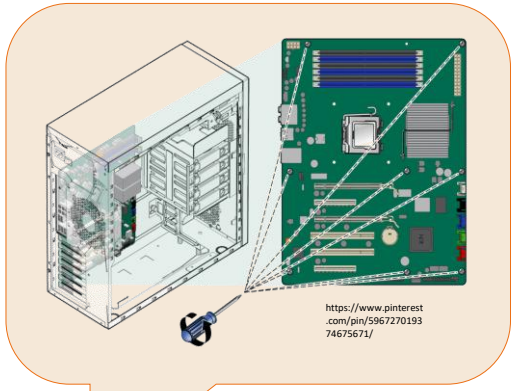


C is indeed a language in high demand in the industry.

Why C?

- Understanding **COMPILERS**, **LINKERS**, **OS**
- **EMBEDDED** (real-time) systems
- Low-level issues in program development:
  - reverse engineering
  - hacking
  - patching
  - direct hardware manipulation

*Studying the underlying system enables us to **write** code that is more **efficient**, more **secure**, and more **reliable**.*

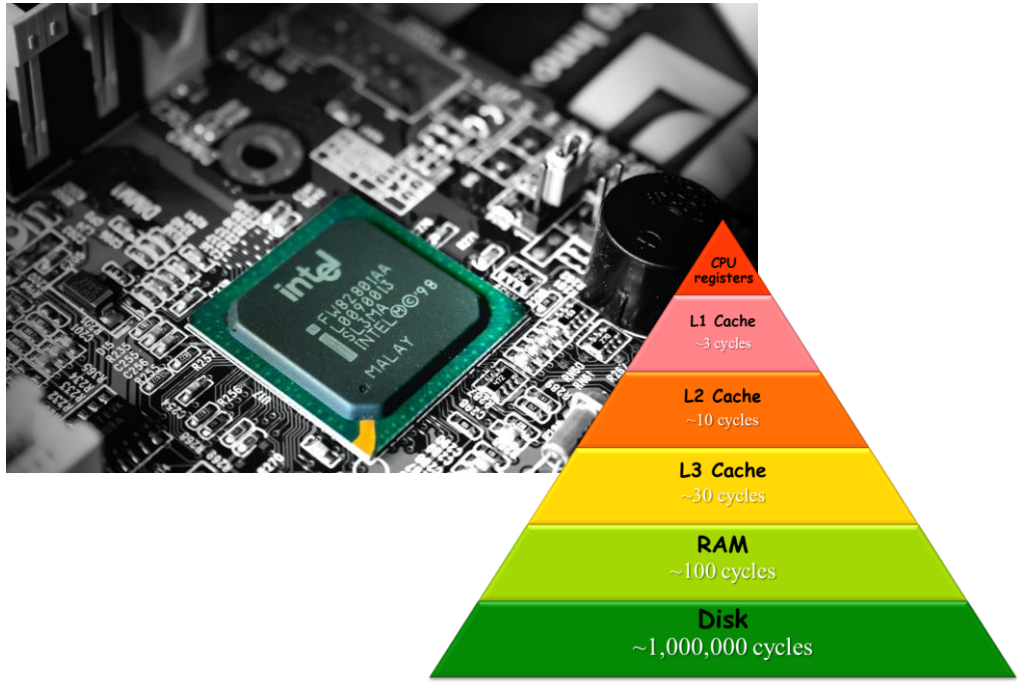


*Think **not just** about the algorithm.*



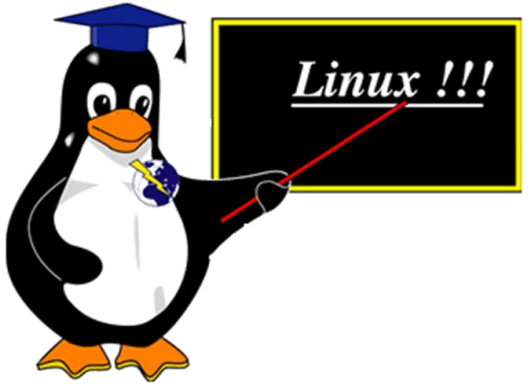
Course Topics

- C programming
- Interaction with OS
- Code optimization
- Cache design and coherence
- C threads
- OpenMP
- GPU architecture and intro to Cuda



Software Tools

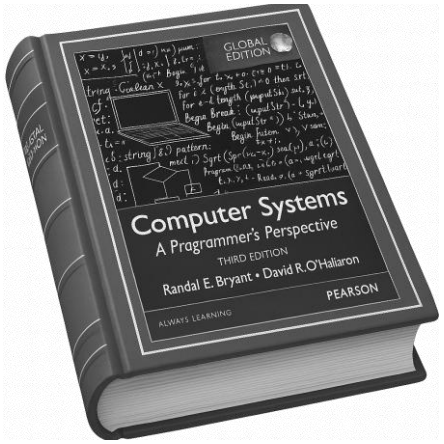
- Linux (no specific distribution, you can try Linux mint)
- **gcc** compiler, **gdb** debugger
- Good programmer's editor or IDE (no preference)
  - **VSCode**



Book

- **Computer Systems: A Programmer's Perspective**

  Bryant and O'Hallaron, Prentice Hall, 3rd edition (2016)



Course Staff

- Lecturer

Dr. Marina Kogan-Sadetsky (sadetsky@bgu.ac.il)

- TA
  - עידן מרדכי גולדברג
  - נור עתאמנה
  - סעיד נעאמנה

- **4 assignments - 20%**
- **final exam – 80%**

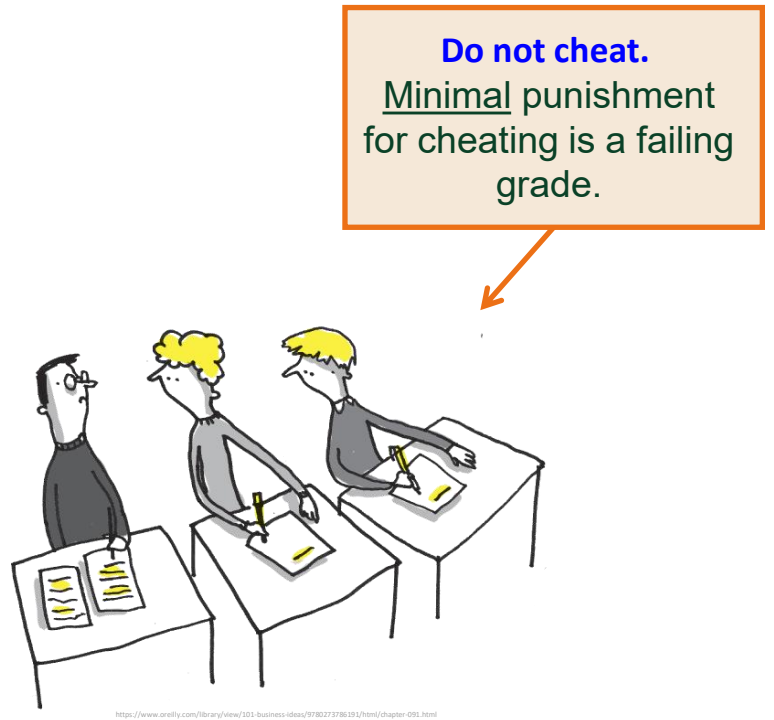
You must pass the assignments (average 56) and pass the exam (56) to pass the course

Extensions policy

- According to the University's guidelines (**תקנון**)
- Each student has a total of **5 grace days**
- If you don't have remaining grace days, late hand-ins will incur 10% penalty for the first day late, additional 20% for the second day late, additional 30% for the third day late.
- No need to notify us
- No bonus if you do not use grace days

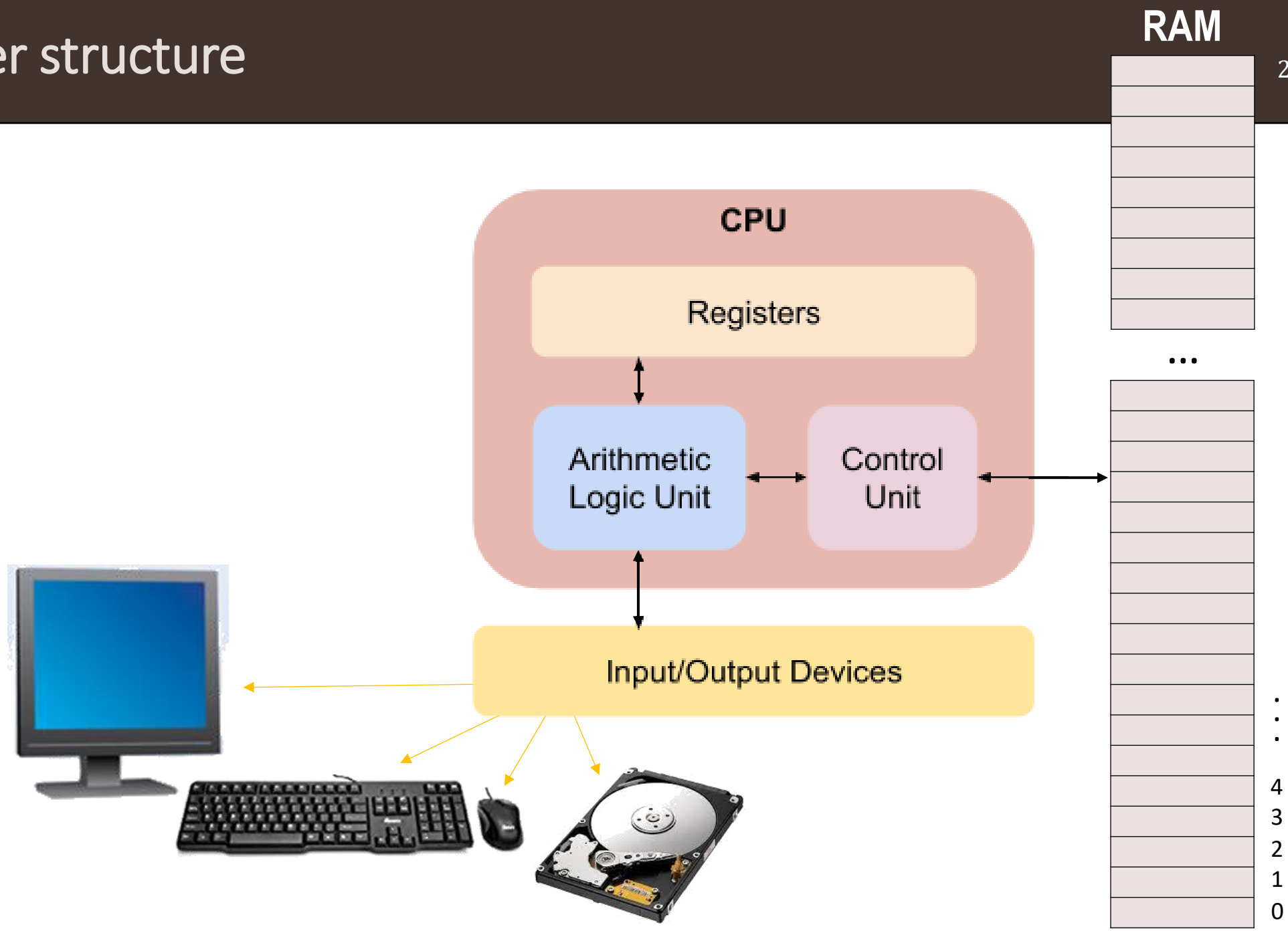
Cheating and Academic Honesty

- **What is cheating?**
  - Sharing code: by copying, retyping, looking at, or supplying a copy of a file
  - Sharing answers to assignments
  - Using AI to solve assignments
- **What is NOT cheating?**
  - Helping others use systems or tools
  - Helping others with high-level code design issues
  - Using AI to debug your code



Let's start

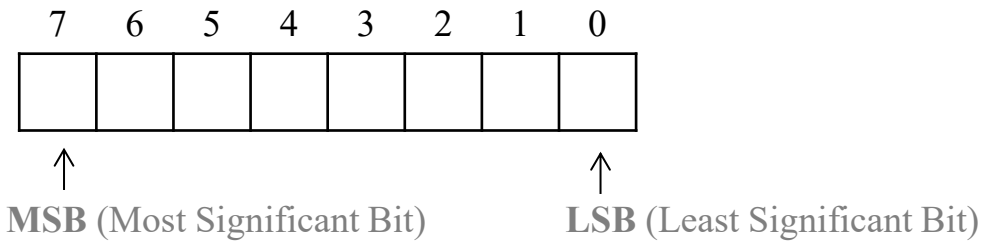
Computer structure



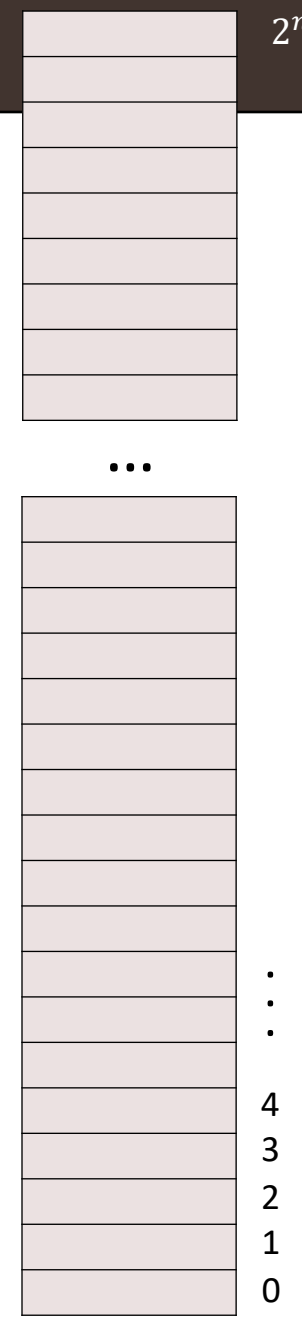
RAM structure

- **bit –** **b**inary dig**it**: 0/1

- **byte –** sequence of 8 bits:



RAM is array of bytes addressed by 0 up to $2^n - 1$



Our First C Program


```c
#include <stdio.h>

int x = 10;
int y;

int main() {
    int z = 5;
    int a[4] = {10, 20, 30, 40};

    printf("z = %d\n", z);

    return 0;
}
```

standard input output C library

global variables – can be accessed from any place in your code

local variables – can be accessed within the scope they are defined

typical way for output

typical way to return from main

Our First C Program


```c
#include <stdio.h>

int x = 10;
int y;

int main() {
    int z = 5;
    int a[4] = {10, 20, 30, 40};

    printf("z = %d", z);

    return 0;
}
```

Questions we usually do not ask:

- Where are **`globals`** stored?
- Where are **`locals`** stored?
- How is **an `array`** stored?

A Variable is a Place in Memory


```
int x = 5;
```

We usually think:

x has the value 5

But in memory:

Address	Value	Name
0x7ffd1000	...	
0x7ffd1004	5	x
0x7ffd1008	...	

```
printf("%d\n", x);
// 5

printf("%p\n", &x);
// 0x7ffd1004
```

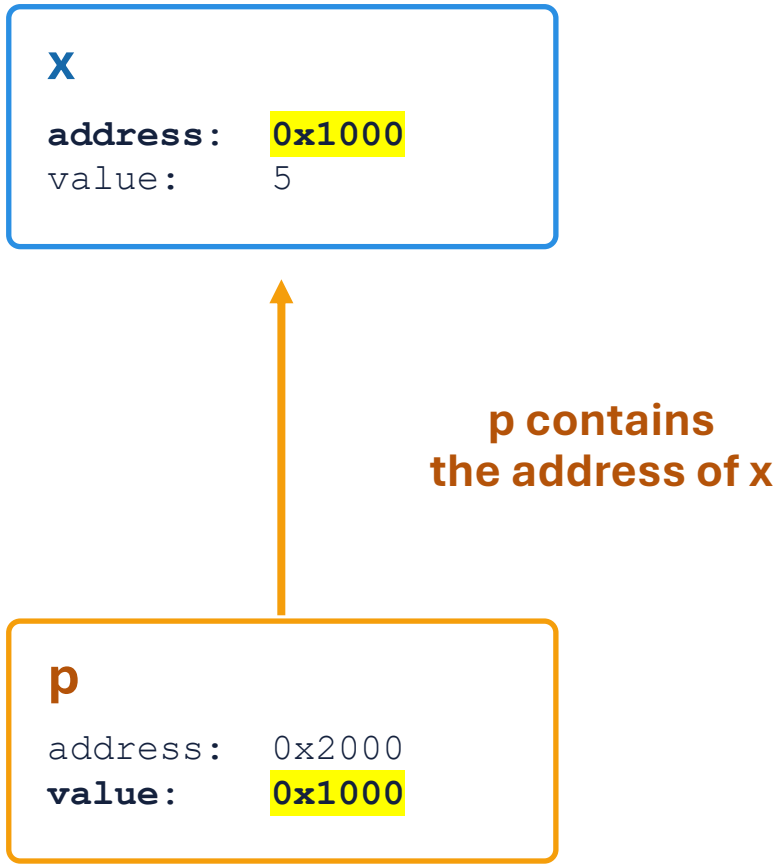
Every object in memory has both a value and an address.

A Pointer Stores an Address


```
int x = 5;
int *p = &x;
```

```
printf("%p\n", p);
printf("%p\n", &x);
```

p == &x



A pointer is a variable whose value is an address.

Dereferencing: Access Through an Address


```
int x = 5;
int *p = &x;

*p = 100;
```

`*p` means: **access the value stored at the address contained in p.**

```
printf("%d\n", x);    // 100
printf("%d\n", *p);   // 100
```

x = value of x &x = address of x p = address stored in p *p = value at that address

Arrays Are Contiguous


```
int a[4] = {10, 20, 30, 40};
```

```
printf("%p\n", &a[0]);
printf("%p\n", &a[1]);
printf("%p\n", &a[2]);
```

Address	Value	Element
0x1000	**10**	a[0]
0x1004	**20**	a[1]
0x1008	**30**	a[2]
0x100C	**40**	a[3]

The elements of an array are stored one after another in memory.

Contiguous storage is what makes memory-access patterns meaningful for performance.

Pointer Arithmetic Knows the Type


```
int a[4] = {10, 20, 30, 40};
int *p = a;
```

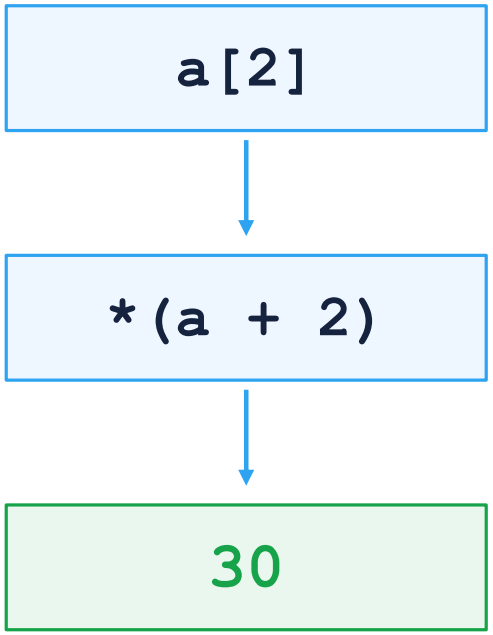
```
p       // 0x1000
p + 1   // 0x1004
p + 2   // 0x1008
```

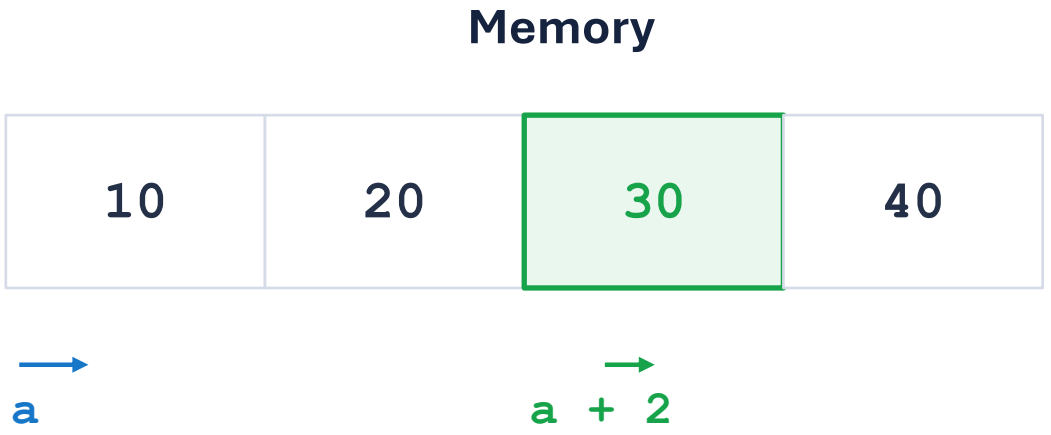
Address	Value	Element
0x1000	**10**	a[0]
0x1004	**20**	a[1]
0x1008	**30**	a[2]
0x100C	**40**	a[3]

p + 1 means: advance by sizeof(*p) bytes

Arrays and Pointers: Closely Related


```
int a[4] = {10, 20, 30, 40};
```





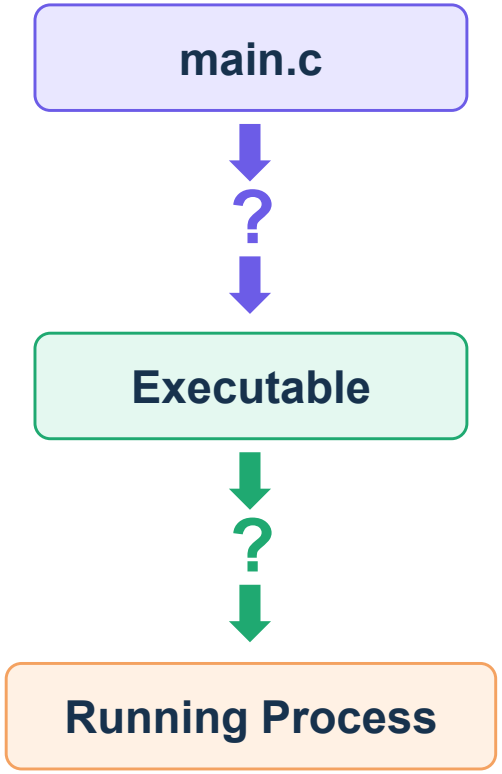
For array access: a[i] is equivalent to *(a + i)

From main.c to a Running Program

main.c

```c
#include <stdio.h>

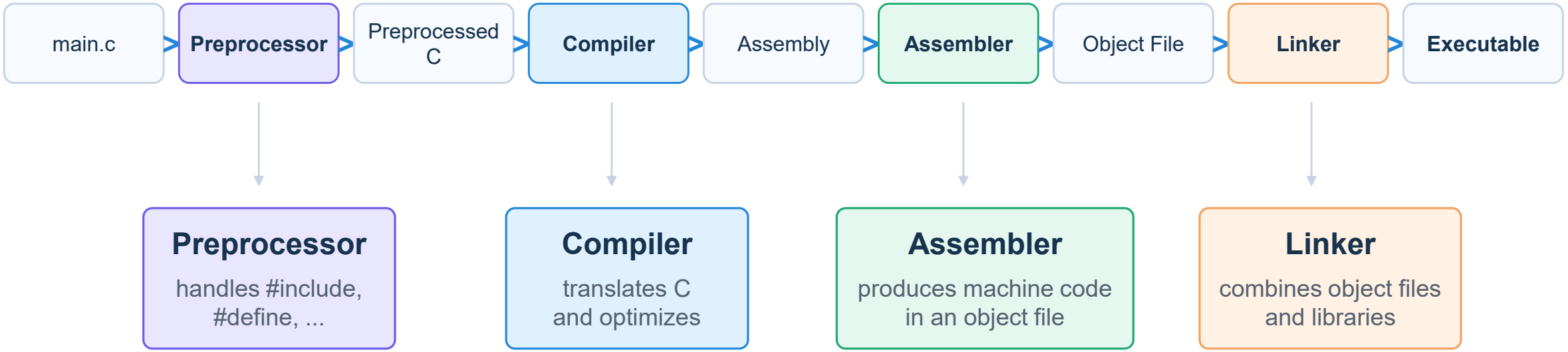
int main() {
    printf("Hello");
    return 0;
}
```



How does source code become an executable?

How does an executable become a running process?

The Compilation Pipeline




```
gcc main.c -o main
```

This one command hides all the stages

The Compilation Pipeline

main.c

```
#include <stdio.h>

int main() {
    printf("Hello");
    return 0;
}
```

gcc

```
.section .rodata
msg: .ascii "Hello"

.section .text
.globl _start
_start:
   mov $1, %rax
   mov $1, %rdi
   lea msg(%rip), %rsi
   mov $5, %rdx
   syscall

   mov $60, %rax
   mov $0, %rdi
   syscall
```

Compiler

translates C
and optimizes

The Compilation Pipeline

main.c

```
#include <stdio.h>

int main() {
    printf("Hello");
    return 0;
}
```

gcc

```
.section .rodata
msg: .ascii "Hello"

.section .text
.globl _start
_start:
   mov $1, %rax
   mov $1, %rdi
   lea msg(%rip), %rsi
   mov $5, %rdx
   syscall

   mov $60, %rax
   mov $0, %rdi
   syscall
```

Assembler

produces machine code in an object file

Linker

combines object files and libraries

gcc

main.exe

```
00001FD0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00001FE0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00001FF0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00002000  01 00 02 00 48 65 6C 6C 6F 00 00 00 01 1B 03 3B  ....Hello......;
00002010  28 00 00 00 04 00 00 00 14 F0 FF FF 84 00 00 00  (...............
00002020  34 F0 FF FF 44 00 00 00 64 F0 FF FF 70 00 00 00  4...D...d...p...
00002030  1A F1 FF FF AC 00 00 00 14 00 00 00 00 00 00 00  ................
00002040  01 7A 52 00 01 78 10 01 1B 0C 07 08 90 01 07 10  .zR..x..........
00002050  10 00 00 00 1C 00 00 00 E8 EF FF FF 22 00 00 00  ............"...
00002060  00 00 00 00 14 00 00 00 00 00 00 00 01 7A 52 00  .............zR.
00002070  01 78 10 01 1B 0C 07 08 90 01 00 00 10 00 00 00  .x..............
00002080  1C 00 00 00 EC EF FF FF 01 00 00 00 00 00 00 00  ................
00002090  24 00 00 00 30 00 00 00 88 EF FF FF 20 00 00 00  $...0....... ...
000020A0  00 0E 10 46 0E 18 4A 0F 0B 77 08 80 00 3F 1A 3B  ...F..J..w...?.;
000020B0  2A 33 24 22 00 00 00 00 1C 00 00 00 58 00 00 00  *3$"........X...
000020C0  66 F0 FF FF 1F 00 00 00 00 41 0E 10 86 02 43 0D  f........A....C.
000020D0  06 5A 0C 07 08 00 00 00 00 00 00 00 04 00 00 00  .Z..............
000020E0  10 00 00 00 01 00 00 00 47 4E 55 00 00 00 00 00  ........GNU.....
000020F0  03 00 00 00 02 00 00 00 00 00 00 00 00 00 00 00  ................
00002100  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00002110  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00002120  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00002130  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
00002140  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Each Compilation Stage Can be Executed Separately


```
gcc main.c -o main
```

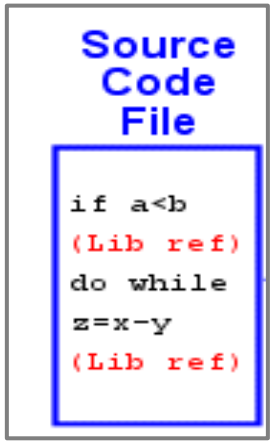
Normally, GCC runs the whole pipeline for us.

`gcc -E main.c -o main.i`	**Stop after preprocessing**
`gcc -S main.c -o main.s`	**Stop after compilation**
`gcc -c main.c -o main.o`	**Stop after assembling**
`gcc main.o -o main`	**Link**

We can stop the pipeline at every stage and inspect the result.

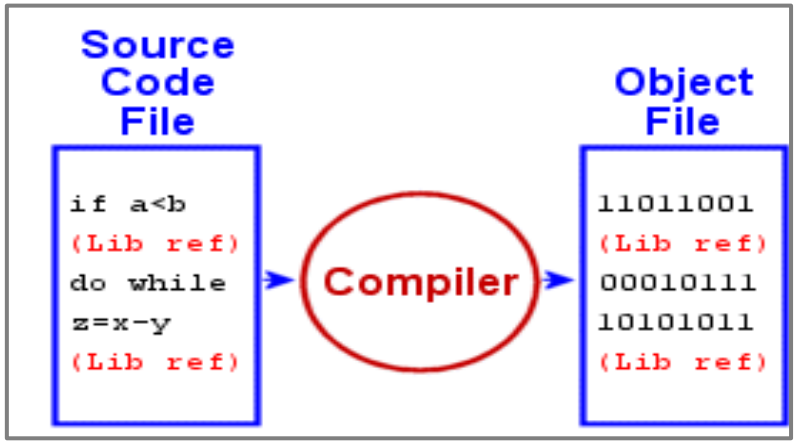
From executable to running process

1. Programmer writes a program in a **SOURCE LANGUAGE**



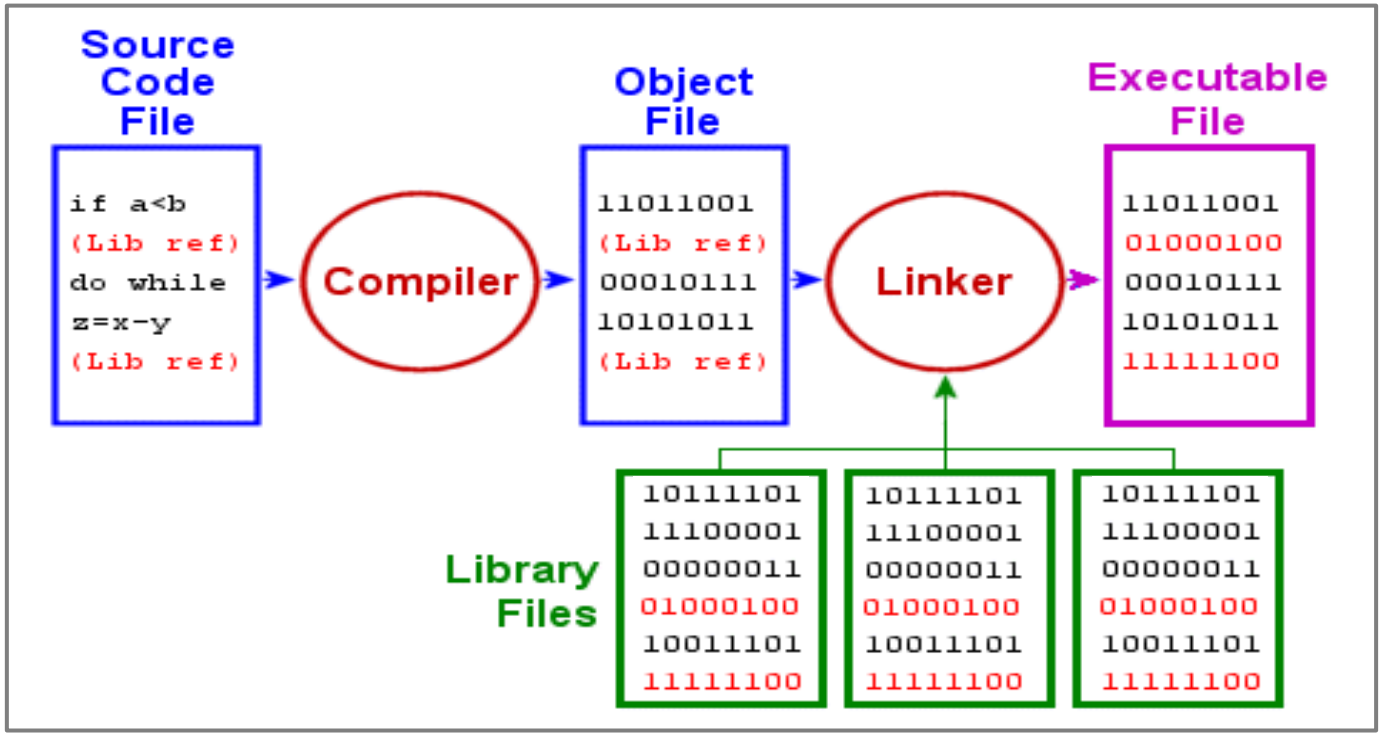
From executable to running process

1. Programmer writes a program in a **SOURCE LANGUAGE**
2. **COMPILER** converts program into **MACHINE LANGUAGE** (object file(s))



From executable to running process

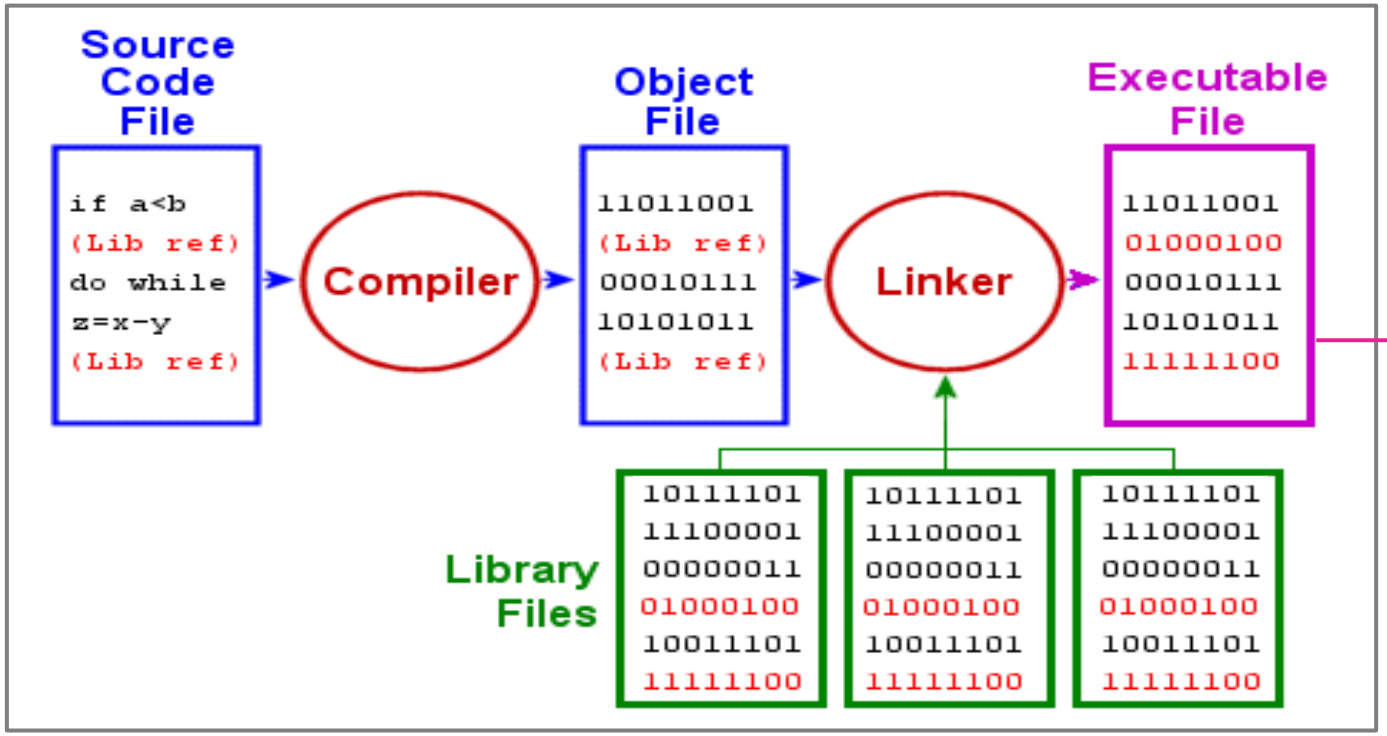
1. Programmer writes a program in a **SOURCE LANGUAGE**
2. **COMPILER** converts program into **MACHINE LANGUAGE** (object file(s))
3. **LINKER** links OBJECT FILE(S) into **EXECUTABLE** file

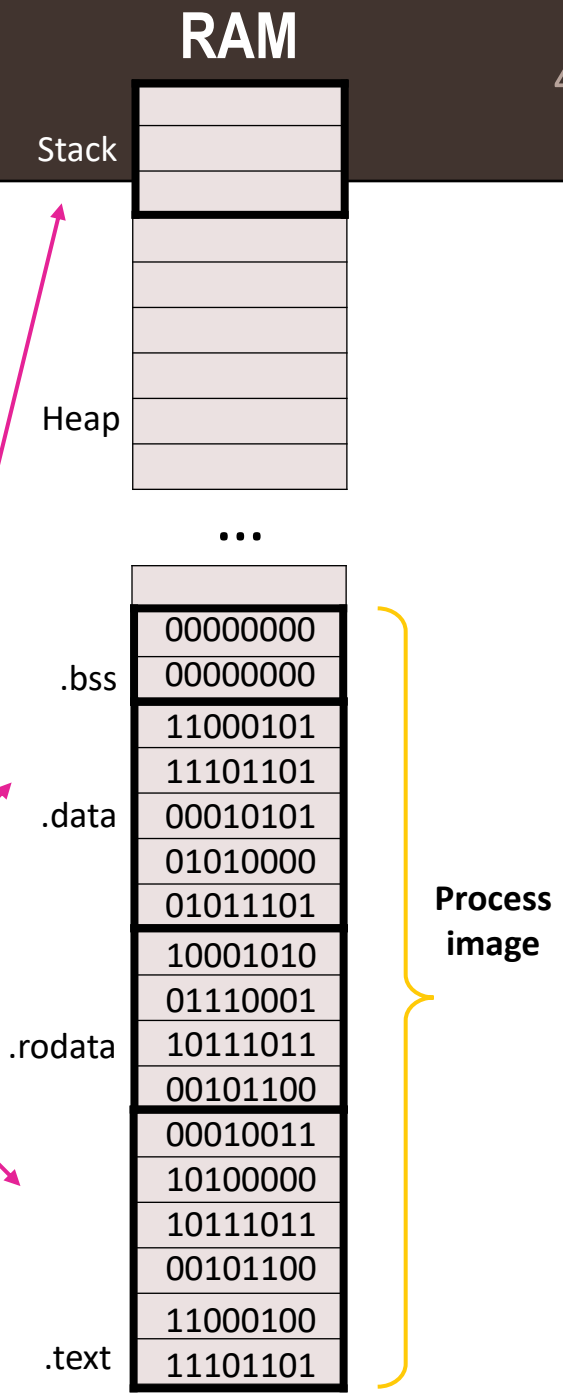


From executable to running process

1. Programmer writes a program in a **SOURCE LANGUAGE**
2. **COMPILER** converts program into **MACHINE LANGUAGE** (object file(s))
3. **LINKER** links OBJECT FILE(S) into **EXECUTABLE** file
4. **LOADER** loads code and data from EXECUTABLE file into RAM

   Initial Stack is allocated





From executable to running process


```c
#include <stdio.h>

int x;

int y = 10;

const int z = 5;

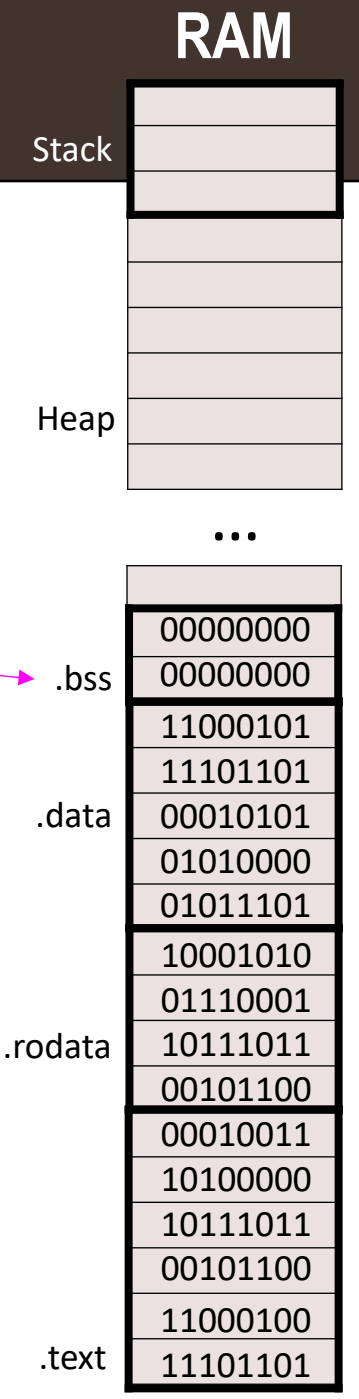
int main() {
    int a = 12;

    printf("z = %d", z);
    return 0;
}
```

bss – **b**lock **s**tarted by **s**ymbol –contains global variables that are initialized to zero, or do not have explicit initialization in source code.

data contains global and static variables that are initialized with non-zero values

rodata – **r**ead **o**nly **data** - contains immutable global variables that are initialized with non-zero values, string literals and constants stored by the compiler



From executable to running process


```c
#include <stdio.h>

int x;

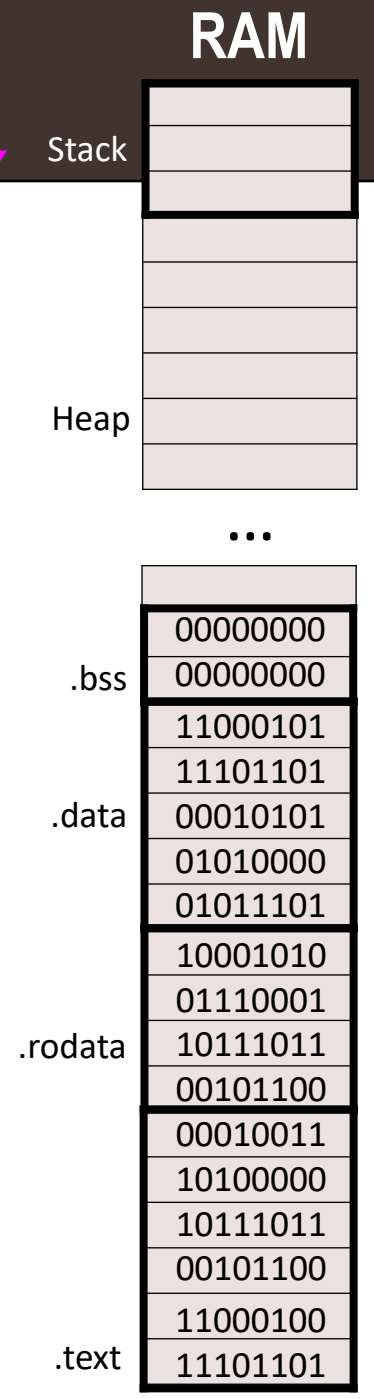
int y = 10;

const int z = 5;

int main() {
    int a = 12;

    printf("z = %d", z);
    return 0;
}
```

local variables are allocated in **Stack**



From executable to running process


```c
#include <stdio.h>

int x;

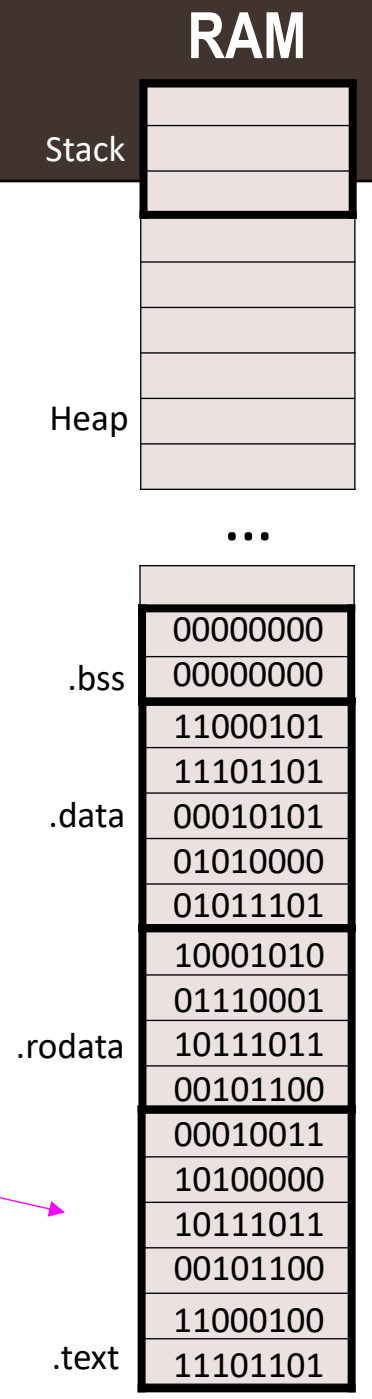
int y = 10;

const int z = 5;

int main() {
    int a = 12;

    printf("z = %d", z);
    return 0;
}
```

text contains the code



From executable to running process


```c
#include <stdio.h>

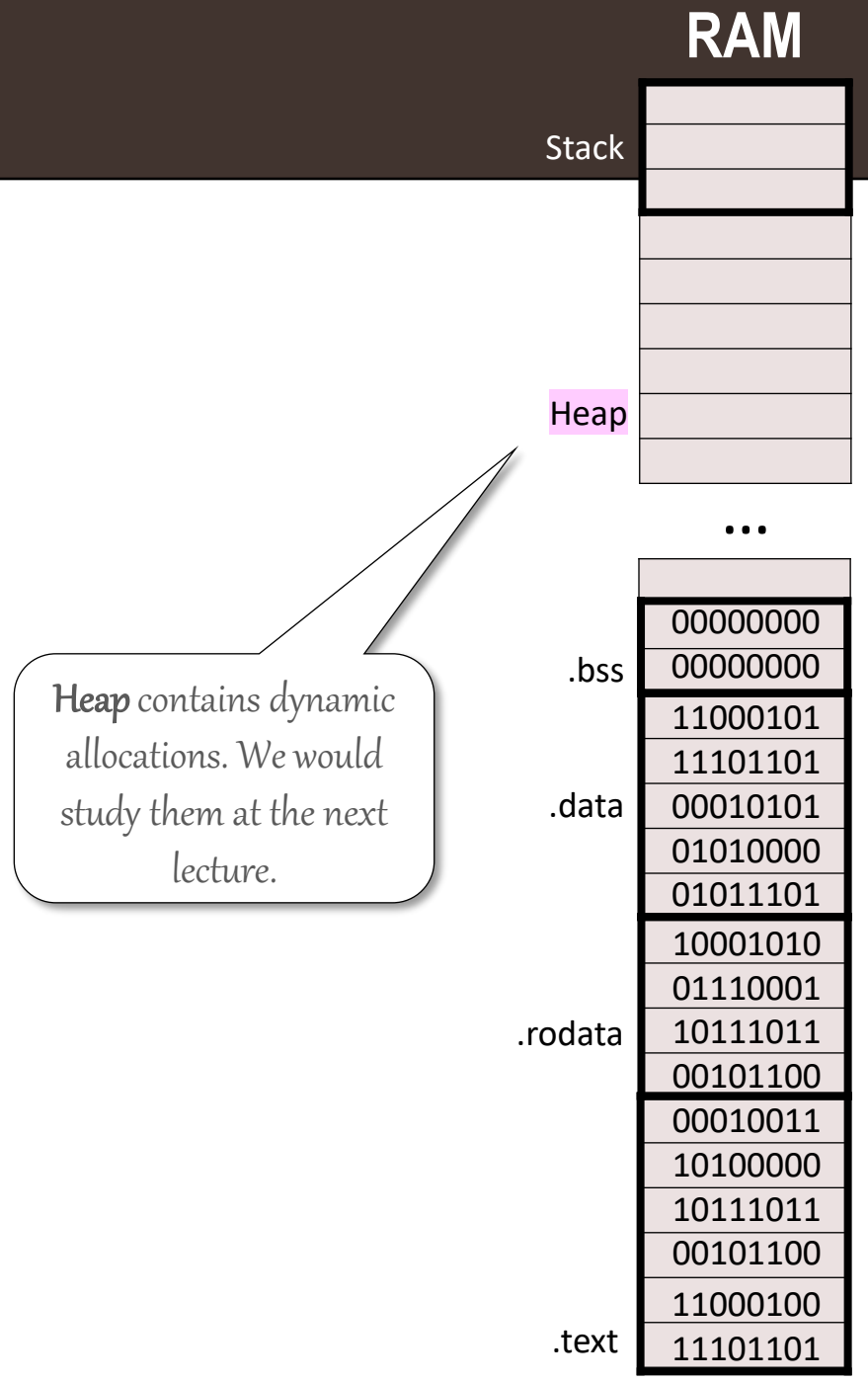
int x;

int y = 10;

const int z = 5;

int main() {
    int a = 12;

    printf("z = %d", z);
    return 0;
}
```



The Process Address Space



What lives where?

Code
program instructions

Global / Static Data
global and static variables

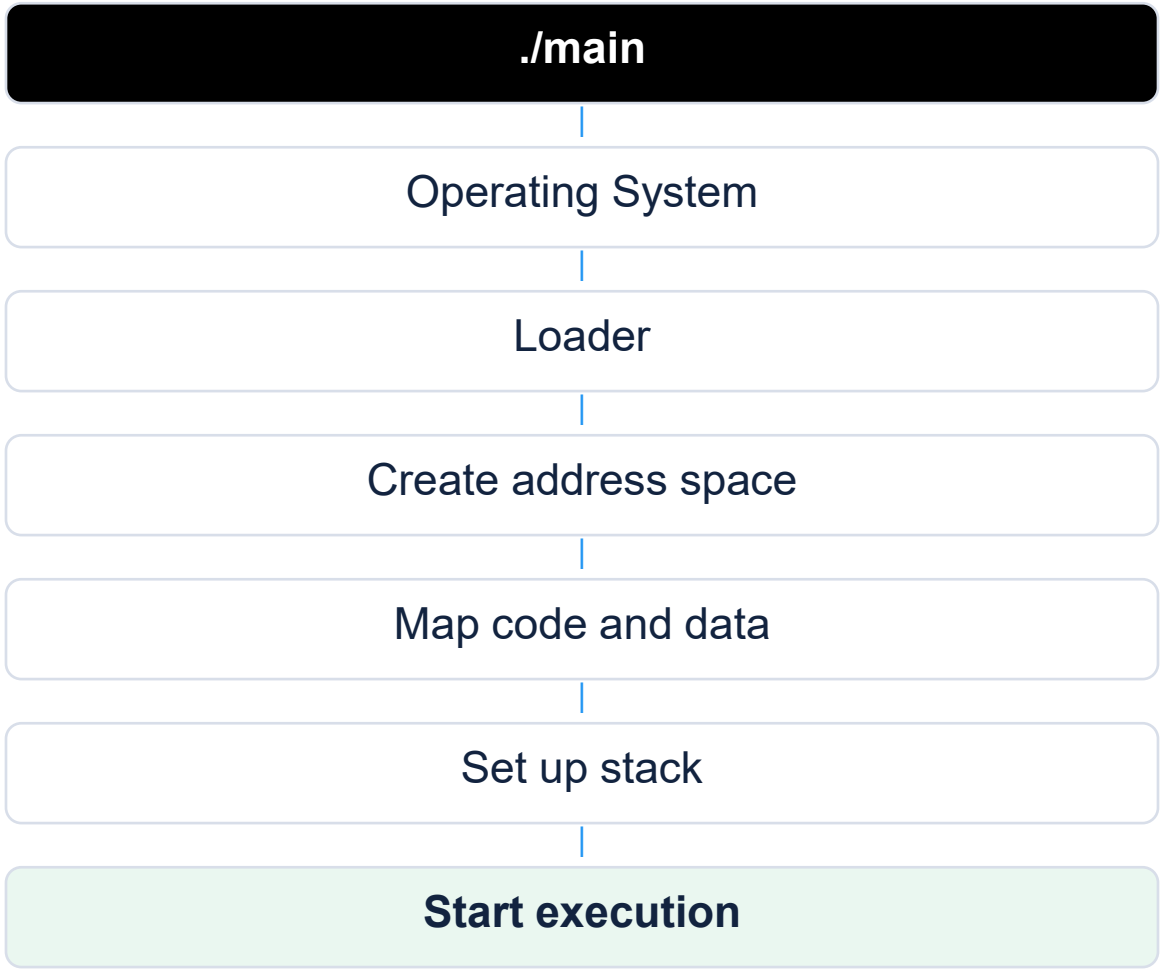
Heap
dynamically allocated memory

Stack
function calls and local variables

Programmer's conceptual view; exact layout can vary.

A process does not just “have memory” — its address space has structure.

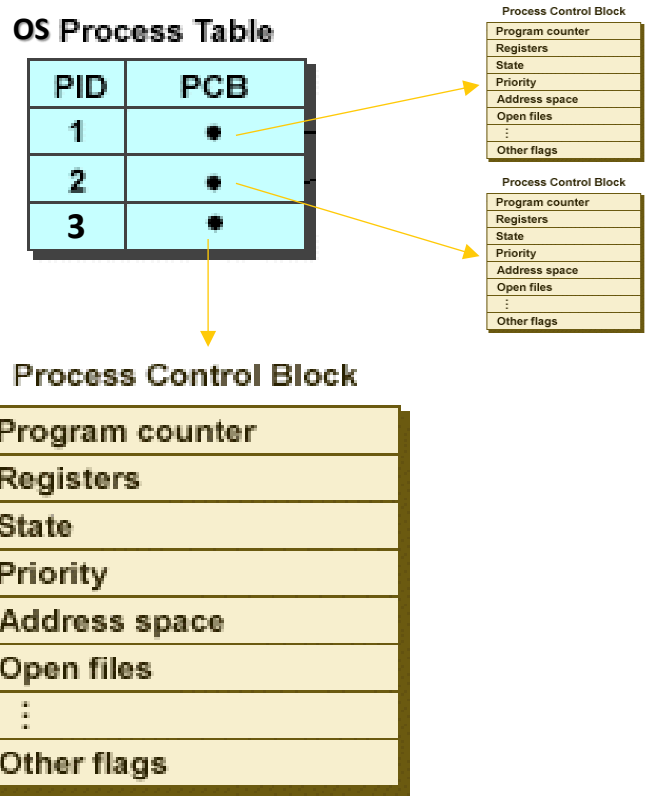
From executable to running process - Summary

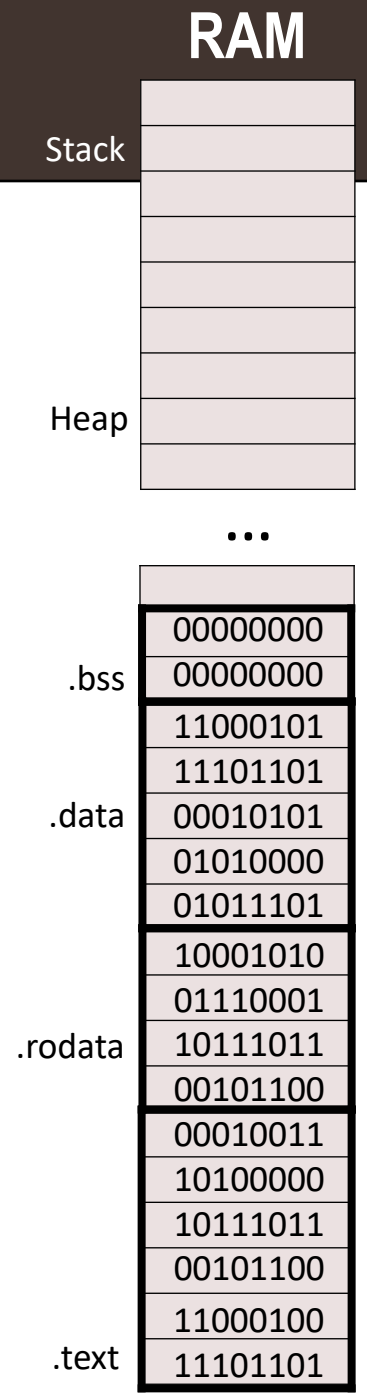


The executable is a file. The OS turns it into a running process.

OS manages the program execution

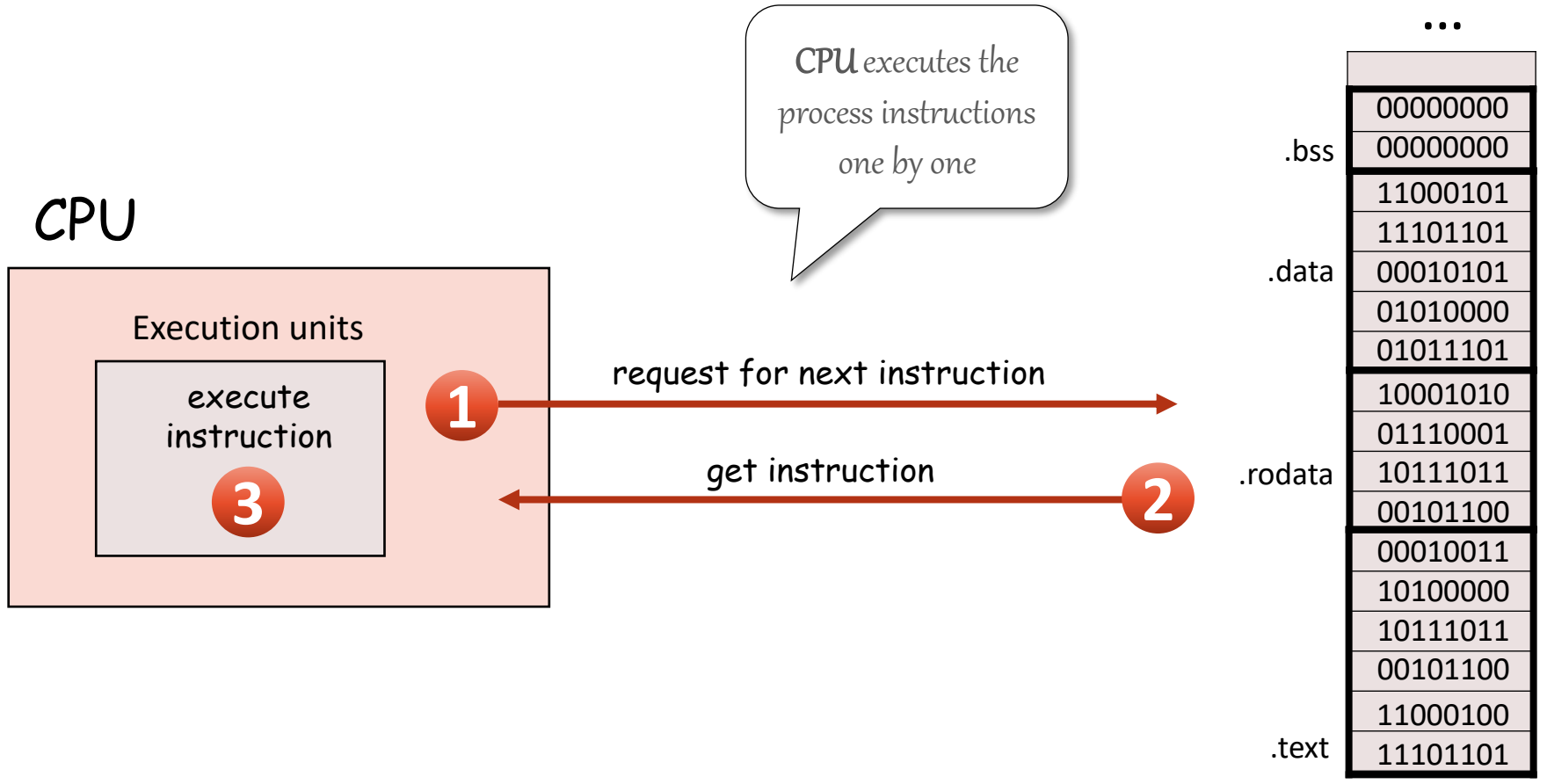
- OS registers the new process in the Process Table
- Process gets PID – Process Identifier.





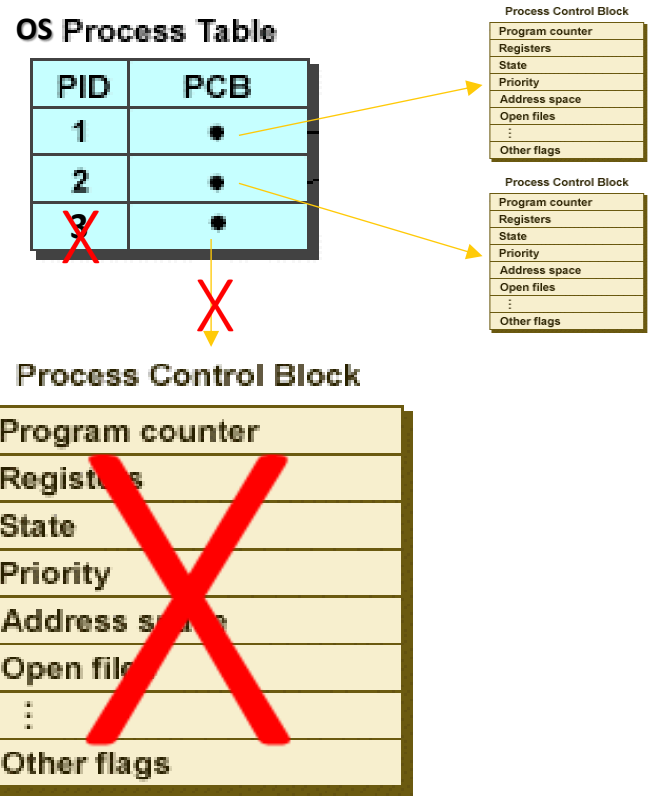
OS manages the program execution

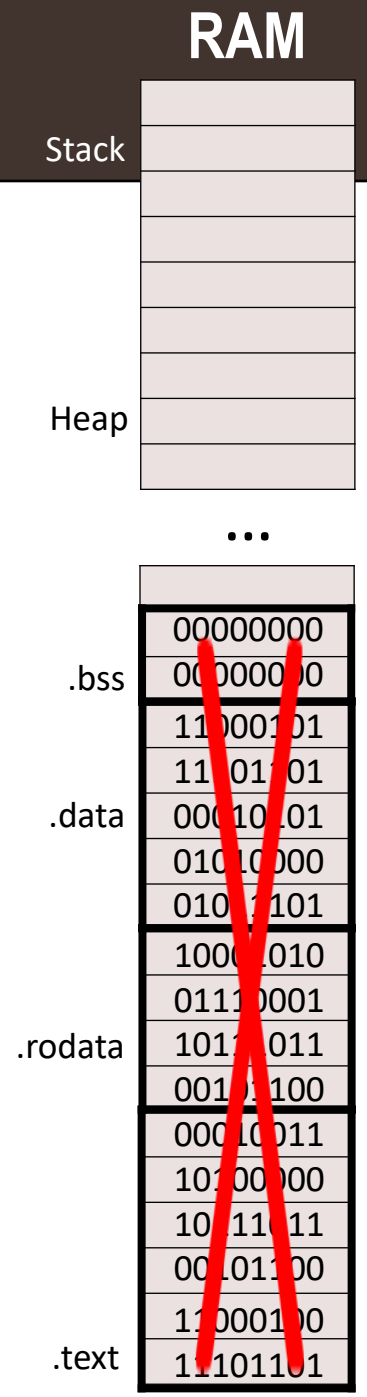
- OS allocated CPU time for a process
- OS serves requests of process
  - additional memory allocations
  - IO devices
  - and so on



OS manages the program execution

- OS terminates the process
  - removes the process from Process Table
  - free the memory allocated for the process



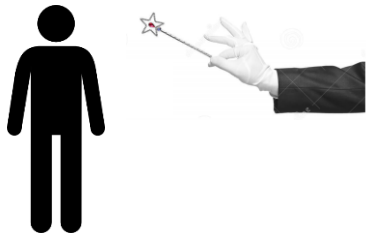


The process lifecycle


```
marina@vm: ~/Parallel
marina@vm:~/Desktop/Parallel$ HelloWorld.exe
Hello world!
marina@vm:~/Desktop/Parallel$
```

creation

OS builds **process image** – allocates memory for the process, and loads its instructions and data from the executable file into the allocated memory



execution

OS identifies the **"main" function** and runs it.

OS manages process' **requests for resources**.



termination

OS **removes** the process and **releases** its resources.



More C basics

Integer Data Types

- **char = 1 byte**
  - char a = 42; // a = 00101010
- **short = 2 bytes**
  - short b = 42; // b = 00000000 00101010
- **int = 4 bytes**
  - int c = 42; // c = 00000000 00000000 00000000 00101010
- **long = 8 bytes**
  - long d = 42; // d = 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00101010

The result of integer calculation is also integer

```
int x = 5;
int y = 2;
x = x / y;     // 5 / 2 = 2
```

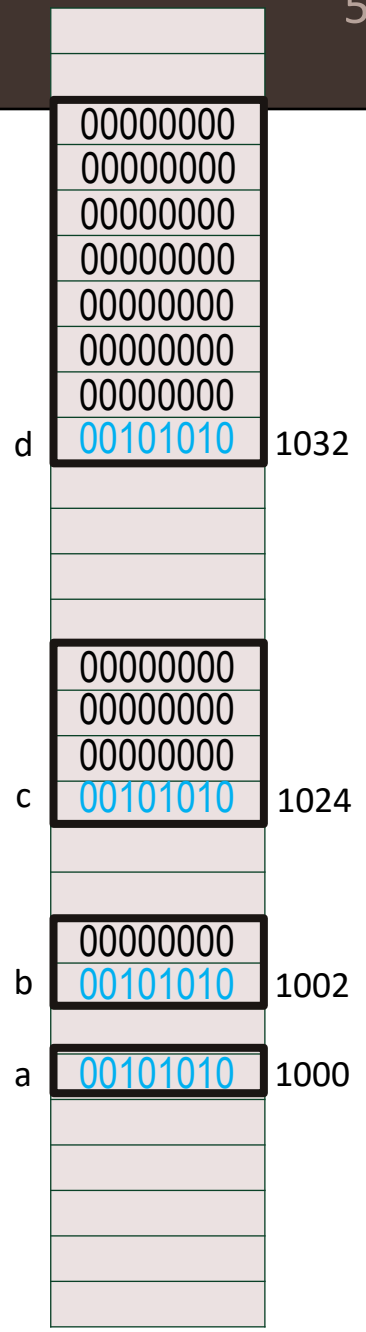
Use char data type for characters, but not only

```
char c1 = 'a';
char c2 = 98;   // equivalent to c2 = 'b'

printf("%c\n", c1);
printf("%d\n", c1);
```

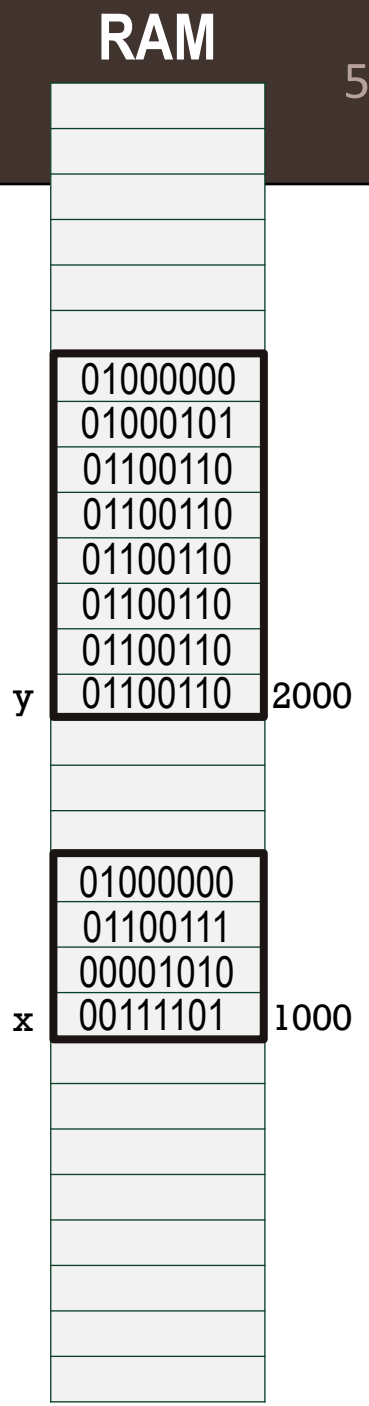
```
a
97
```

The extra leading bits on the left are filled with zeros



Floating-point Data Types

- float = 4 bytes
  - float x = 3.61; // x = 0 10000000 11001110000101000111101
- double = 8 bytes
  - double y = 42.8; // x = 0 10000000100 0101011001100110011001100110011001100110011001100110



This time the division result is (probably) precise.

```
double x = 5.0;
double y = 2.0;
x = x / y; // 5 / 2 = 2.5
```

Why probably precise and not precise for sure?

Sizeof operator

- Unary operator returning the number of bytes required to store its operand
- Can be applied to a variable or a constant.

```c
#include <stdio.h>

int main() {
  short x;
  int y;
  long z;
  printf("%d, %d, %d\n", sizeof(x), sizeof(y), sizeof(z));
  printf("%d\n", sizeof(int));
  return 0;
}
```

returns size of (variable) data type

```
2, 4, 8
4
```

Sizes of data types can vary on different Architectures and Operating Systems. We can use sizeof() to detect the sizes on our computer.

Basic C IO functions


```c
#include <stdio.h>

int main() {
    int x;
    float y;
    char z;

    printf("Enter integer number\n");
    scanf("%d", &x);
    printf("Enter float point number\n");
    scanf("%f", &y);
    printf("Enter character\n");
    scanf(" %c", &z);

    printf("x=%d, y=%f, z=%c\n", x, y, z);

    return 0;
}
```

please note the space before %c

- scanf is used for input
- printf for output

& means address of the variable in RAM

Syntax:

scanf("% format string", &variable name);

printf("% format string", variable name);

you must #include <stdio.h> library to use the I/O functions

```
> gcc main.c
> ./a.out
```

```
Enter integer number
5
Enter float point number
3.5
Enter character
b
x=5, y=3.5, z=b
```



Control Flow

C supplies a set of commands which control the flow of the program

name	purpose
• if-else • switch-case	decision making
• for • while • do-while	loops
• break • continue • return	branching



Structures

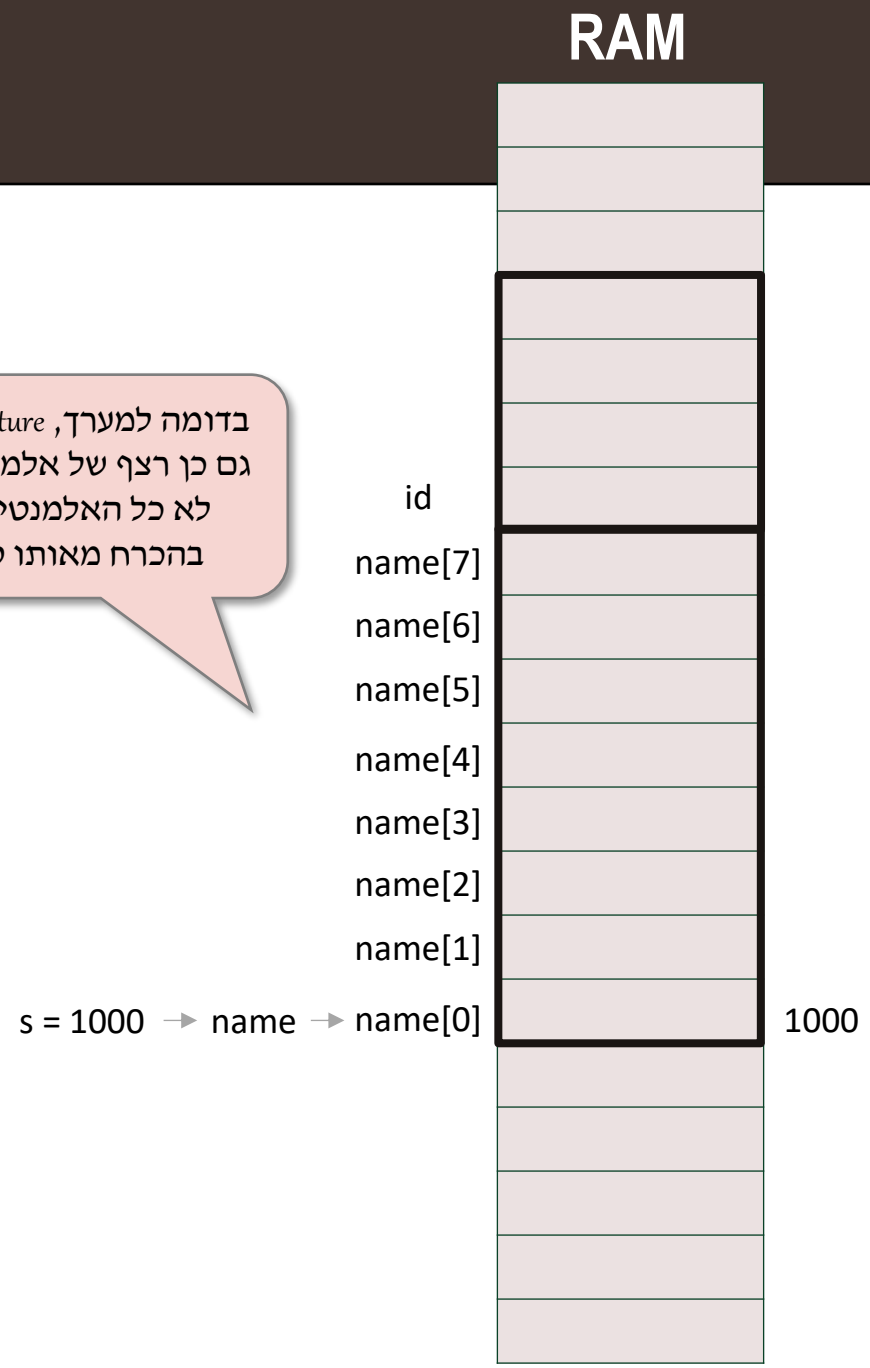
Collection of variables of possibly different types

```
struct Student {
  char name[8];
  int id;
};
```

struct Student is a new data type

בדומה למערך, structure הוא גם כן רצף של אלמנטים, אך לא כל האלמנטים שלו בהכרח מאותו טיפוס

```
int main() {
  struct Student s;
}
```



Structures


```
struct Student {
  char name[8];
  int id;
};
```

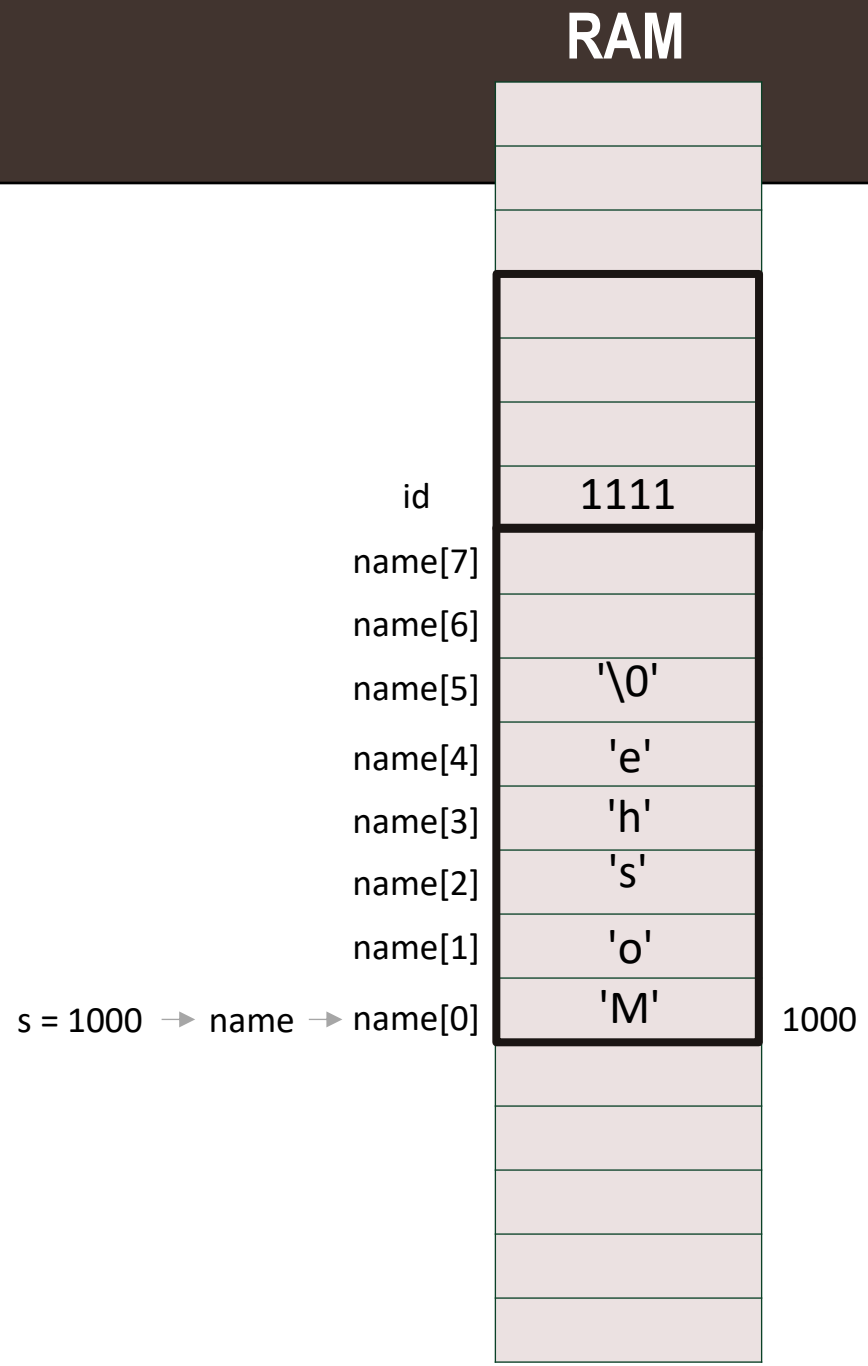
struct Student is a new data type

```
int main() {
  struct Student s = {"Moshe", 1111};
}
```

We can initialize the fields via the definition.

The initialization parameter list corresponds to the structure's fields – in the same order!

"Moshe" defines a string - an array of characters with '\0' character at the end)

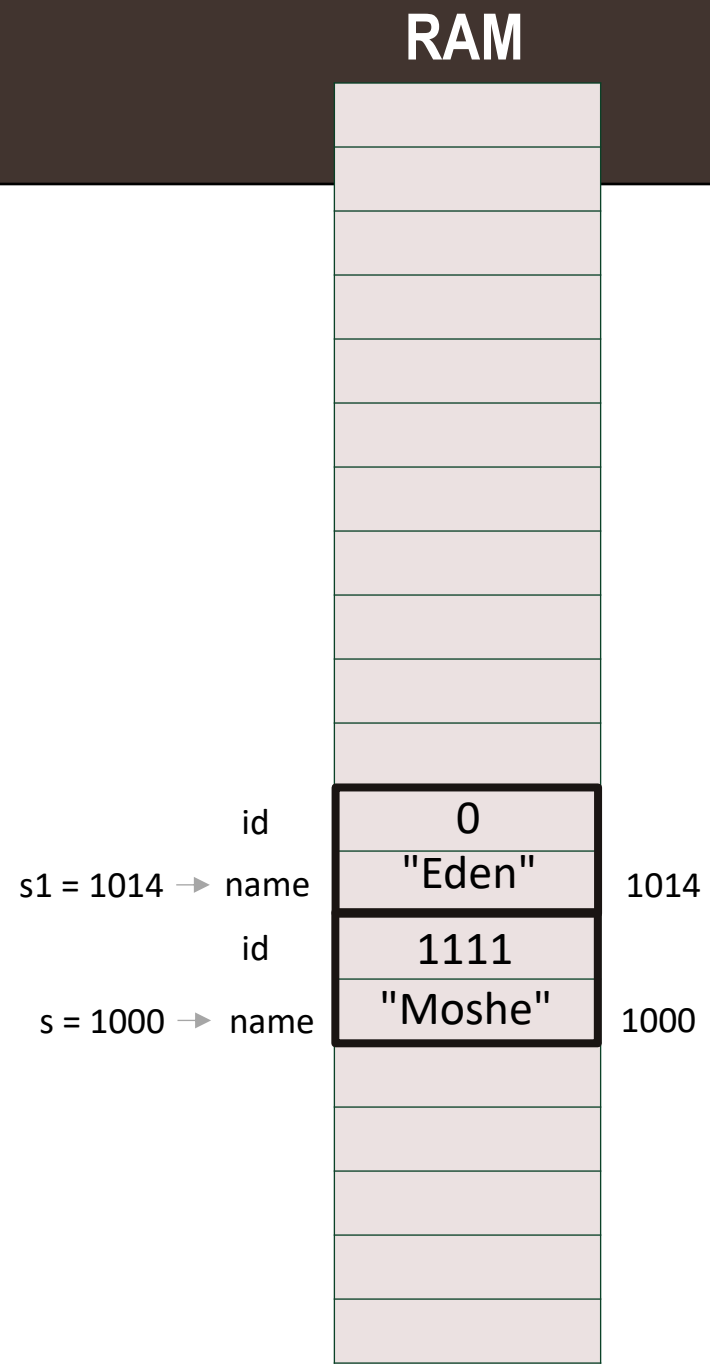


Structures


```
struct Student {
  char name[8];
  int id;
};
```

```
int main() {
  struct Student s = {"Moshe", 1111};
  struct Student s1 = {"Eden"};  // defines {"Eden", 0}
}
```

Missing initial values would be completed to be 0

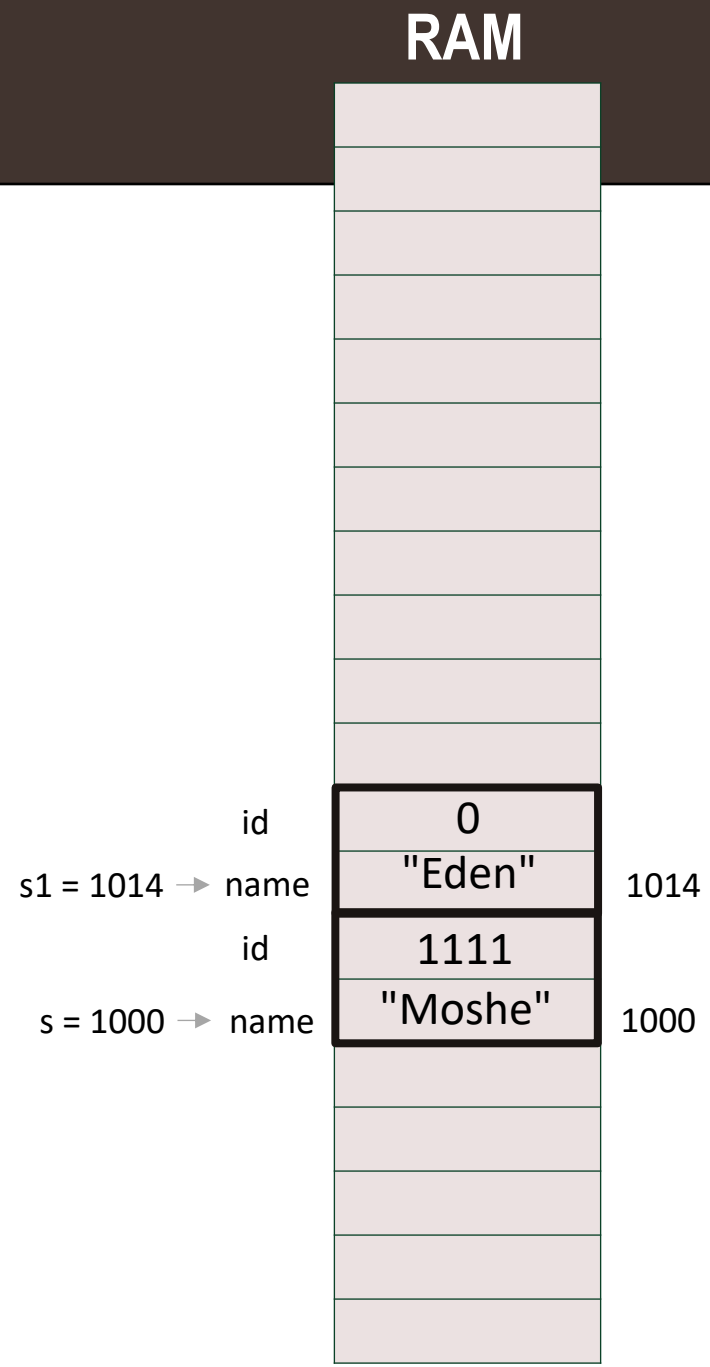


Structures

```
struct Student {
  char name[8];
  int id;
};
```

```
int main() {
  struct Student s = {"Moshe", 1111};
  struct Student s1 = {"Eden"};  // defines {"Eden", 0}
  struct Student s2 = {2222, "Omer"};  // compilation error
}
```

The initialization parameter list corresponds to the structure's fields – in the same order!



Structures


```
typedef struct {
   char name[8];
   int id;
}Student;
```

an alternative definition

we do not need to use 'struct' keyword anymore

```
int main() {
   Student s = {"Moshe", 1111};
   struct Student s1 = {"Omer", 2222}; // compilation error
}
```

indeed, the previous syntax is unavailable

```
typedef struct Student{
   char name[8];
   int id;
}Student;
```

we can combine both definitions together

now we can use both ways

```
int main() {
   Student s = {"Moshe", 1111};
   struct Student s1 = {"Omer", 2222};
}
```

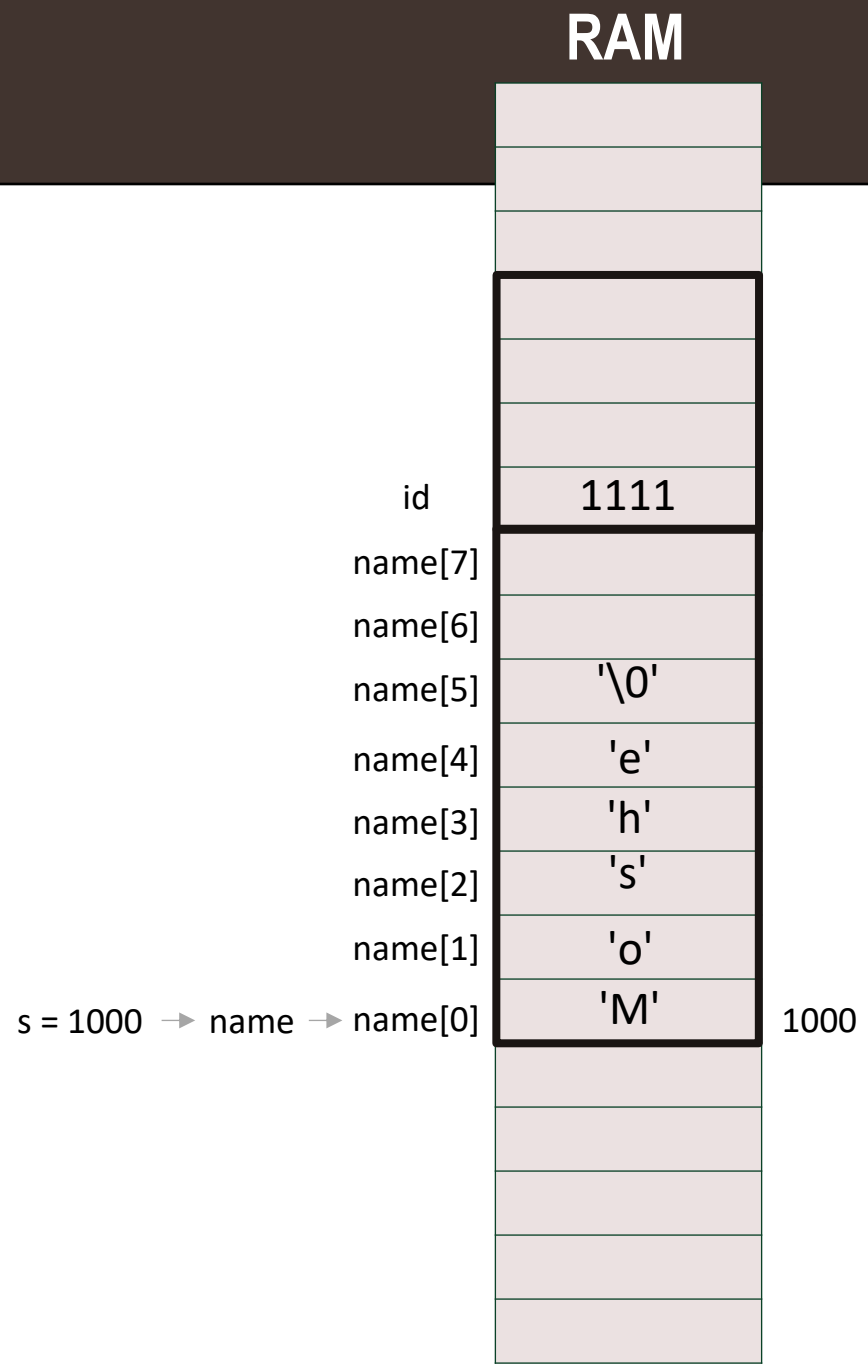
Structures


```
typedef struct {
   char name[8];
   int id;
}Student;
```

```
int main() {
   Student s = {"Moshe", 1111};
   printf("%s, %d", s.name, s.id);
}
```

we use '.' operator to access structure fields

'%s' is string printing (string is an array of characters with '\0' character at the end)



Structures

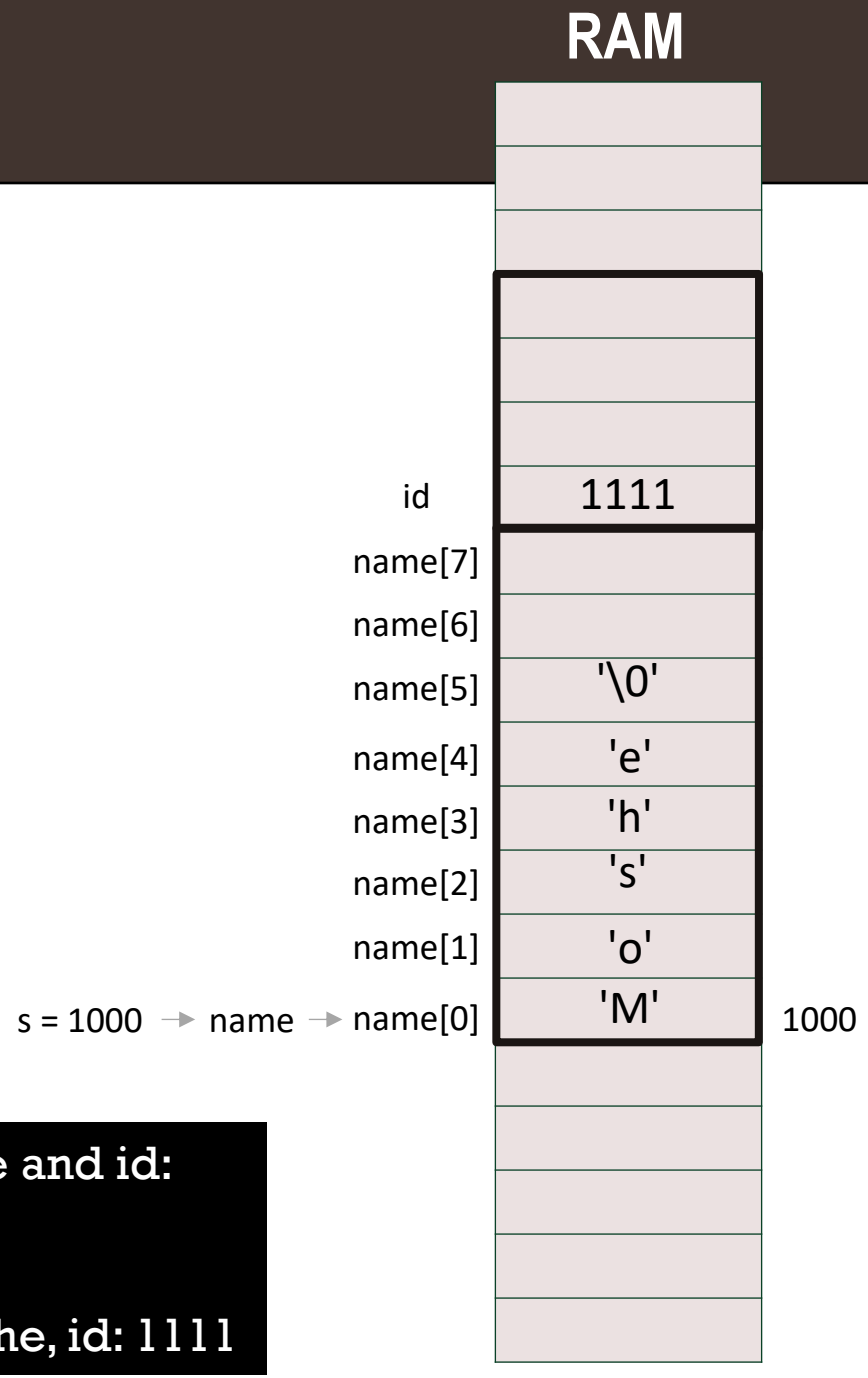
```
typedef struct {
   char name[8];
   int id;
}Student;
```

We can initialize the structure fields separately of its definition

since the name of array is its address, we do not use '&' on this field in scanf

```
int main() {
   Student s;
   printf("Enter name and id: ");
   scanf("%s %d", s.name, &(s.id));
   printf("name: %s, id: %d", s.name, s.id);
}
```

```
Enter name and id:
Moshe
1111
name: Moshe, id: 1111
```



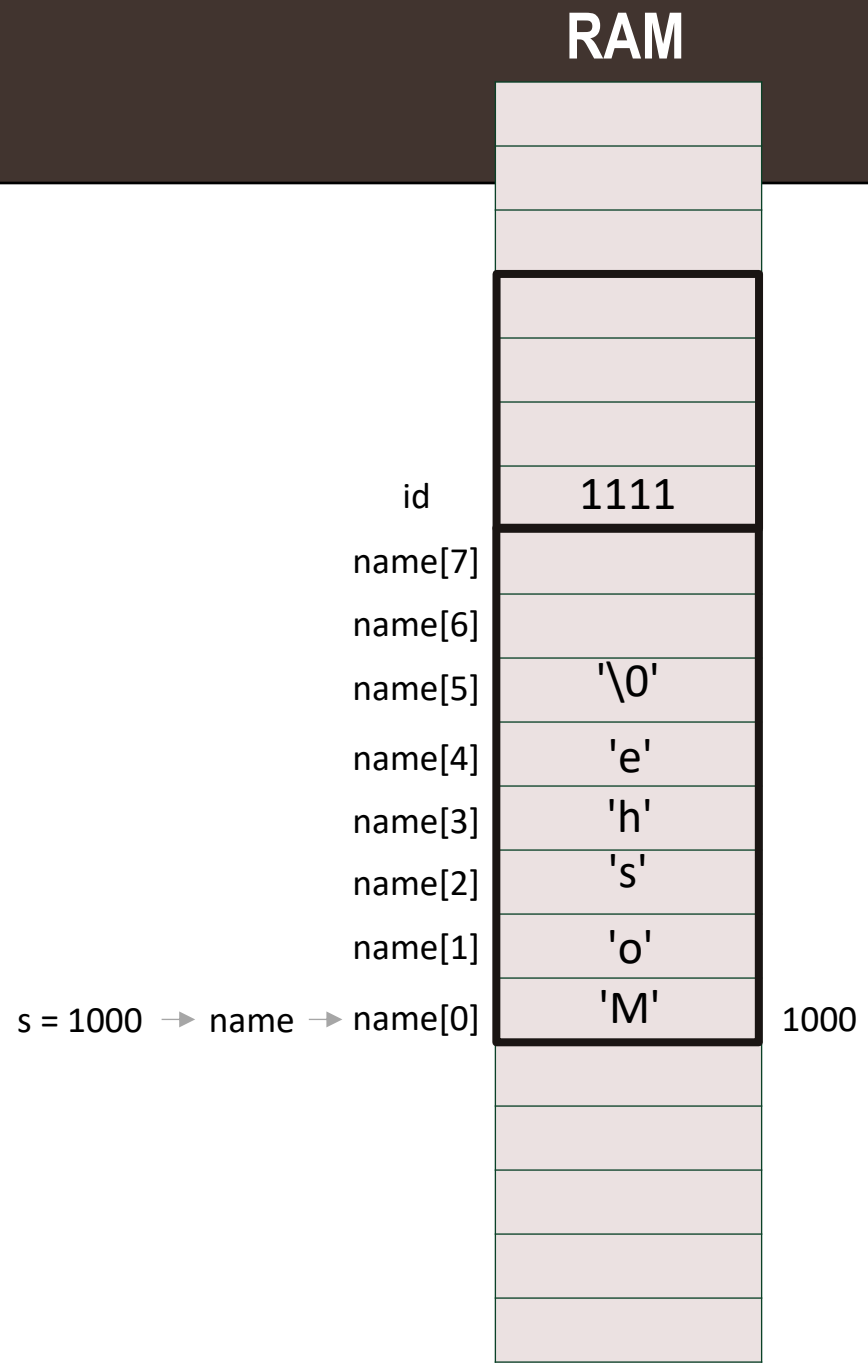
Structures

```
struct Student {
   char name[8] = "Moshe";
   int id = 1111;
};  // compilation error
```

We cannot initialize fields in struct declaration

```
struct S {
   int x;
   static int y;
};  // compilation error
```

struct type cannot have static members



Command-line Arguments

self-training


```
marina@vm: ~/Parallel
marina@vm:~/Parallel$ gcc main.c
marina@vm:~/Parallel$ ./a.out Omer 2 3
Hi Omer
2+3=5
marina@vm:~/Parallel$
```

The arguments that are passed via the command line to the program are called 'command-line arguments". They are passed to the 'main' function.

'main' can get two following arguments:

argc – an integer specifying how many arguments are passed to the program via the command line

argv – an array of pointers to the argument list

But it seems like 'main' does not get any arguments ...

```
int main() {
    ...
    return 0;
}
```

```
int main(int argc, char *argv []) {
    ...
    return 0;
}
```

argv is indeed an array of pointers to char

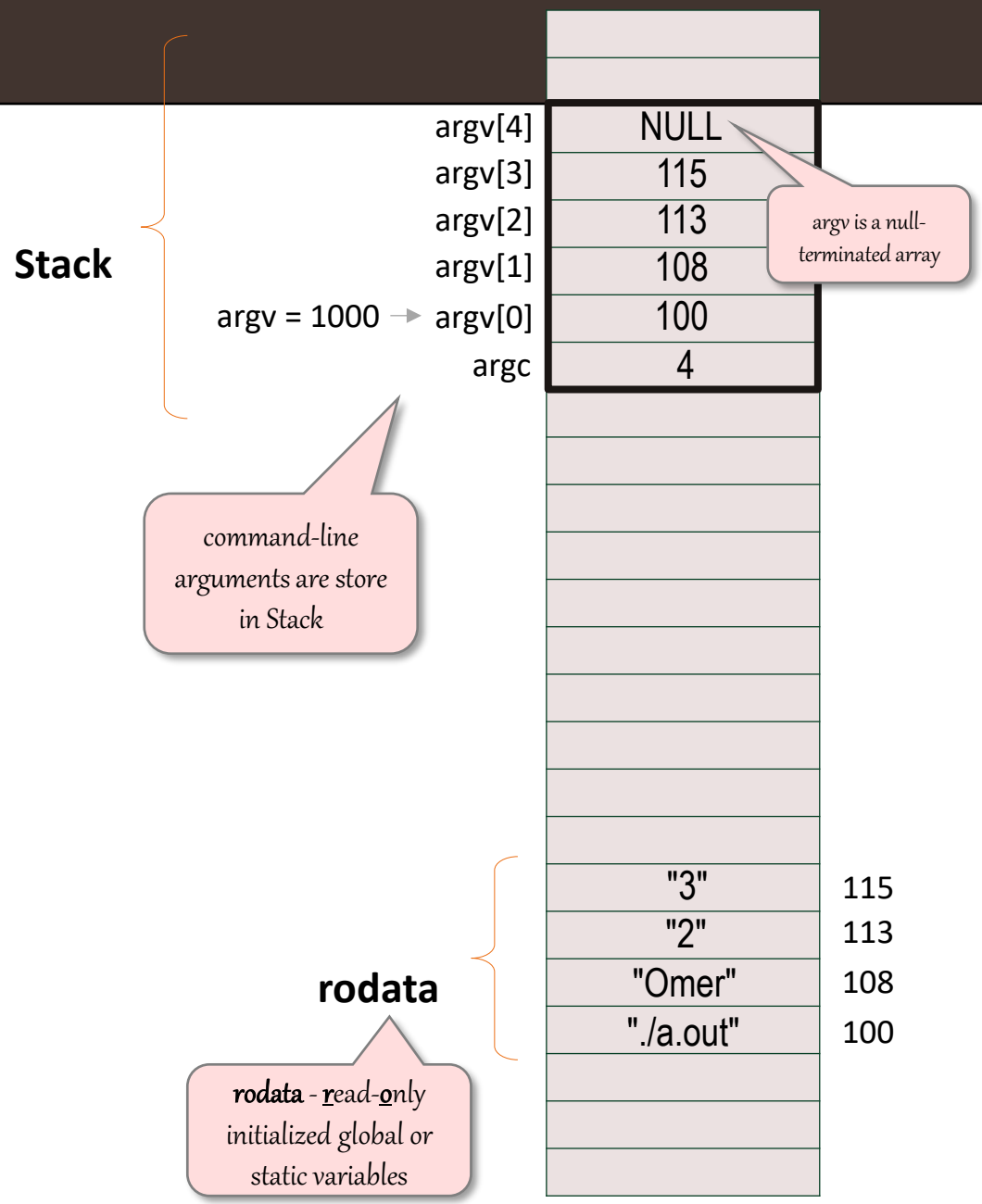
Command-line Arguments

self-training


```
marina@vm: ~/Parallel
marina@vm:~/Parallel$ gcc main.c
marina@vm:~/Parallel$ ./a.out Omer 2 3
Hi Omer
2+3=5
marina@vm:~/Parallel$
```

```
int main(int argc, char *argv []) {
    ...
    return 0;
}
```

Let us write the program code



Command-line Arguments

self-training


```
marina@vm: ~/Parallel
marina@vm:~/Parallel$ gcc main.c
marina@vm:~/Parallel$ ./a.out Omer 2 3
Hi Omer
2+3=5
marina@vm:~/Parallel$
```

use %s format to print strings

```c
#include <stdio.h>
#include <stdlib.h>

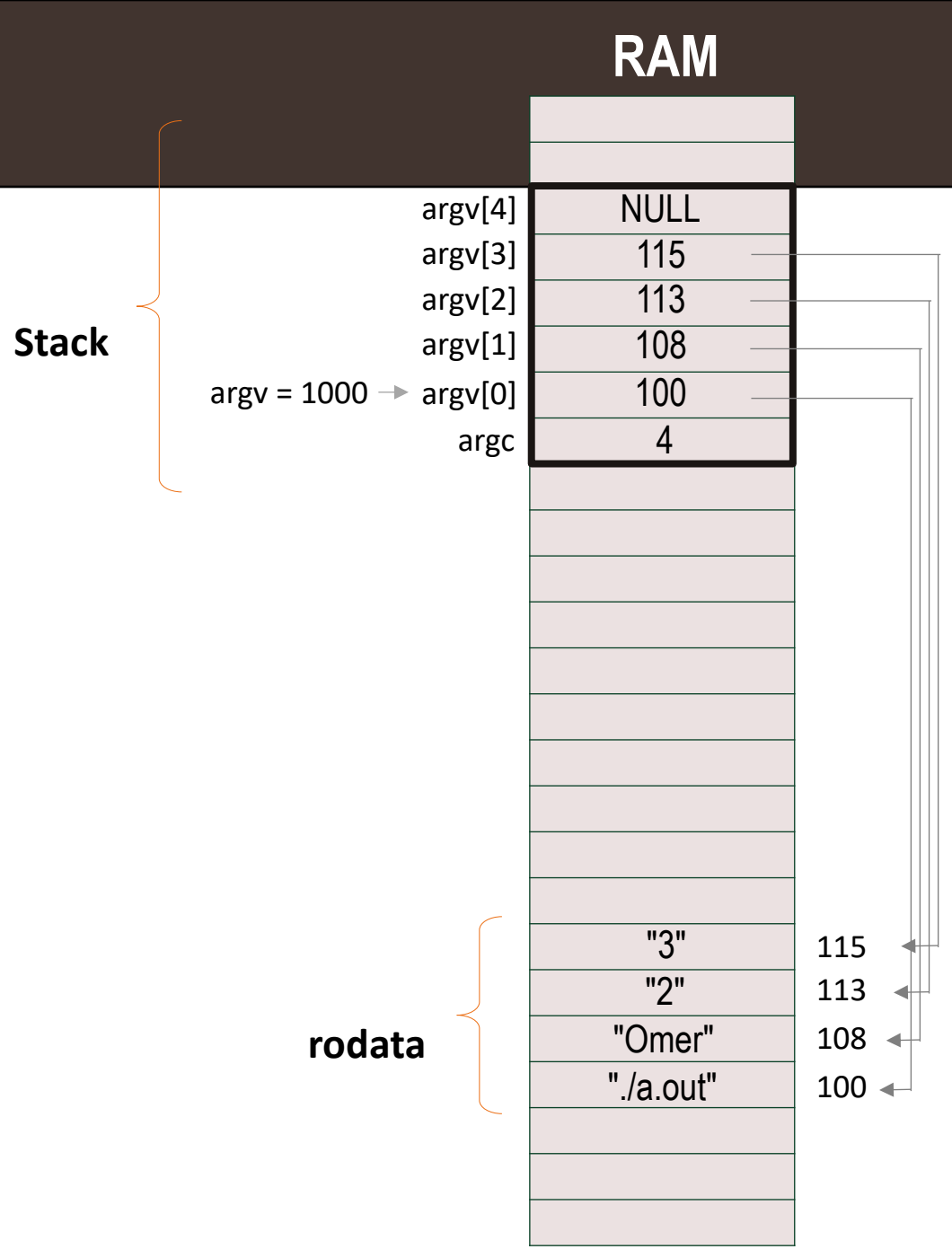
int main(int argc, char *argv []) {
    if (argc != 4)
        return 1;
    printf("Hi %s\n", argv[1]);

    int x = atoi(argv[2]);  // converts "2" to 2
    int y = atoi(argv[3]); // converts "3" to 3
    printf("%d+%d=%d\n", x, y, x+y);

    return 0;
}
```

atoi – **a**scii **to** **i**nteger – is a C standard library function that converts string to integer. The string must represent valid integer number.

Hi Omer
2+3=5



Summary — What Did We Learn Today?

- **Why Low-Level Programming Matters** – Understanding hardware helps us write faster and more reliable code.
- **C and Memory** – Variables, addresses, pointers, arrays, and structures.
- **Data Representation** – How different data types are represented and stored in memory.
- **From Source Code to Executable** – Compilation, assembly, and linking.
- **From Executable to Running Process** – How the OS loads and manages a program.
- **C Programming Basics** – Data types, sizeof, input/output, and structures.

Key Takeaway: To write efficient code, we need to understand what happens beneath the code.

Next Time


```
int *p = malloc(100 * sizeof(int));

...

free(p);
```

malloc **calloc** **realloc** **free**

Who owns allocated memory?

How long does it live?

What happens if we forget to release it?

What happens if we access it after it was released?

In C, memory management is part of the program.

Quiz!

self-training

SANITY TEST



```
struct st {
    int x;
    int y;
};

int main() {
    printf("%d", sizeof(struct st));
    return 0;
}
```



What is the output of the program ?

(A) 4
(B) 8
(C) Compiler Error
(D) Runtime Error



self-training

sizeof(struct st) = sizeof(int + int) = 8 bytes

```
struct st {
   int x;
   int y;
};

int main() {
   printf("%d", sizeof(struct st));
   return 0;
}
```

8

What is the output of the program ?

(A) 4
(B) 8
(C) Compiler Error
(D) Runtime Error

self-training

SANITY TEST



```
struct number {
   float x;
};

int main() {
 struct number n1, n2, n3;
   n1.x = 4;
   n2.x = 3;
   n3 = n1 + n2;
   printf("%f", n3.x);
   return 0;
}
```

???

What is the output of the program ?

(A) 7
(B) some unpredictable number
(C) Compiler Error
(D) Runtime Error

self-training

SANITY TEST



```
struct number {
   float x;
};

int main() {
 struct number n1, n2, n3;
   n1.x = 4;
   n2.x = 3;
   n3 = n1 + n2;
   printf("%f", n3.x);
   return 0;
}
```

This is illegal operation.

How to fix this code ?

What is the output of the program ?

(A) 7
(B) some unpredictable number
(C) Compiler Error
(D) Runtime Error

```
error: invalid operands to binary +
(have ‘struct number’ and ‘struct number’)
```

self-training

SANITY TEST



```
struct number {
   float x;
};

int main() {
 struct number n1, n2, n3;
   n1.x = 4;
   n2.x = 3;
   n3.x = n1.x + n2.x;
   printf("%f", n3.x);
   return 0;
}
```

7

What is the output of the program ?

(A) 7
(B) some unpredictable number
(C) Compiler Error
(D) Runtime Error

self-training

SANITY TEST



Write a program that defines 3 students, each has name, id, faculty, and grades for 5 exams. Initialize the students' data with user's input values.

???

self-training

SANITY TEST



Write a program that defines 3 students, each has name, id, faculty, and grades for 5 exams. Initialize the students' data with user's input values.

```
typedef struct {
   char name [10];
   char faculty [5];
   int id;
   int grades [5];
}Student;

int main() {
   Student students[3];
   for(int i=0; i<3; i++) {
      printf("Enter name, faculty and id: ");
      scanf("%s%s%d", students[i].name, students[i].faculty, &(students[i].id));
      printf("Enter five grades: ");
      for(int j=0; j<5; j++)
         scanf("%d", &(students[i].grades[j]));
  }
   return 0;
}
```

Thank you !

